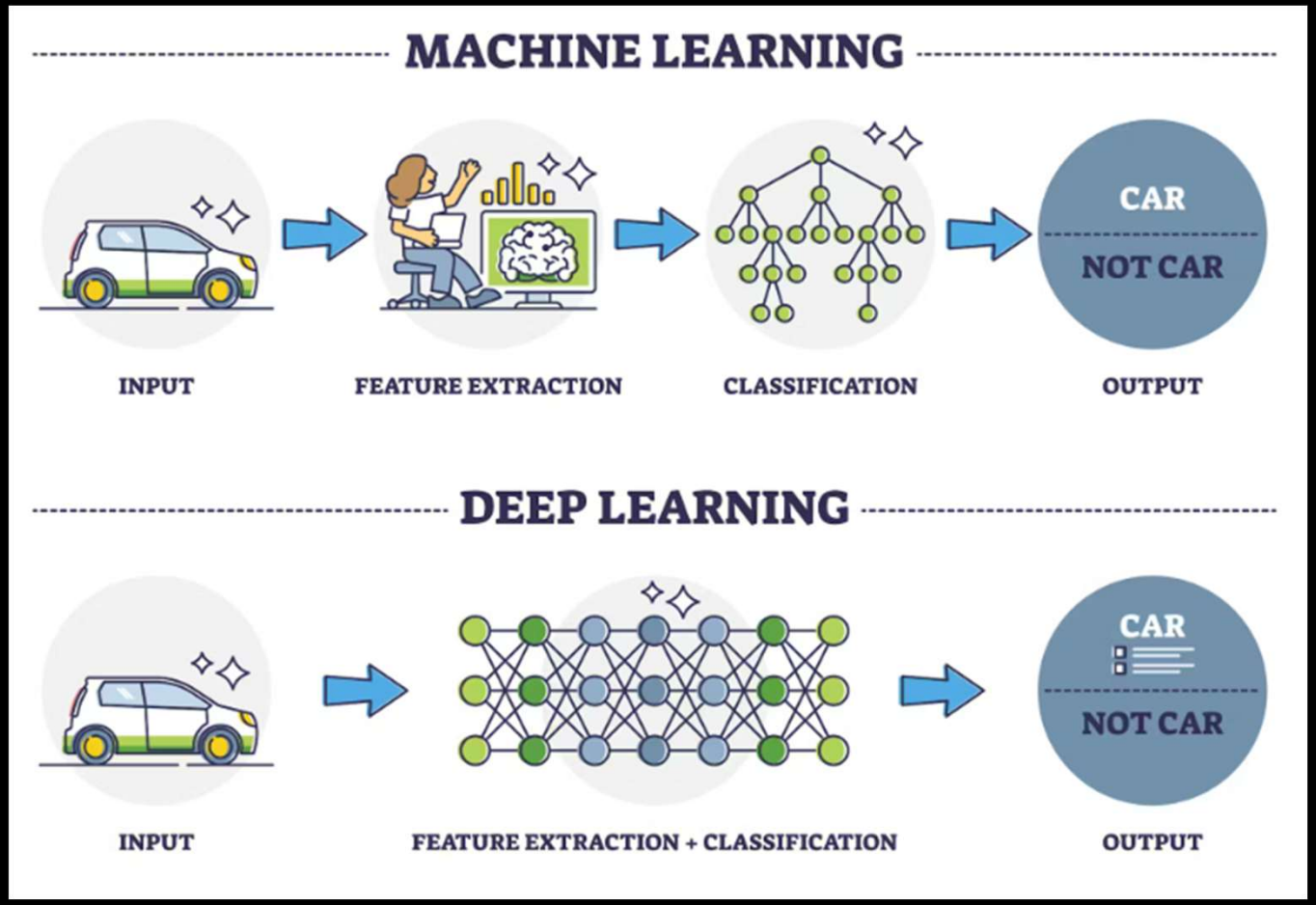
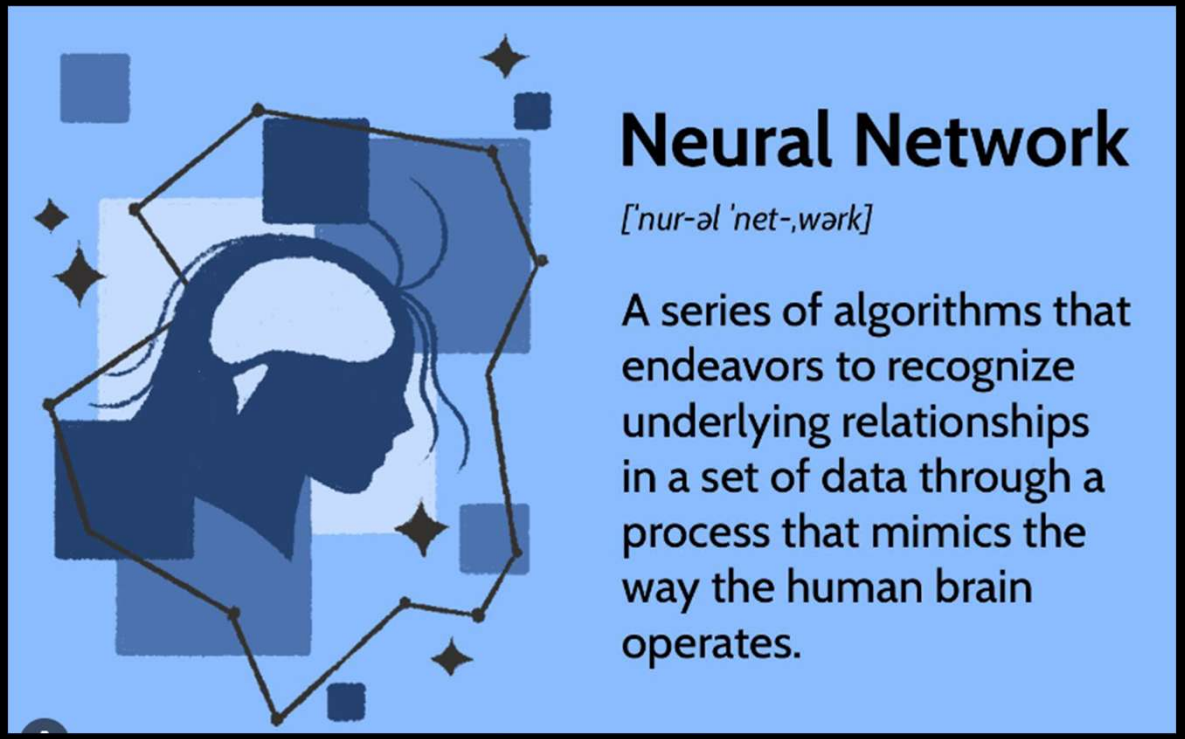
Generative AI Academy

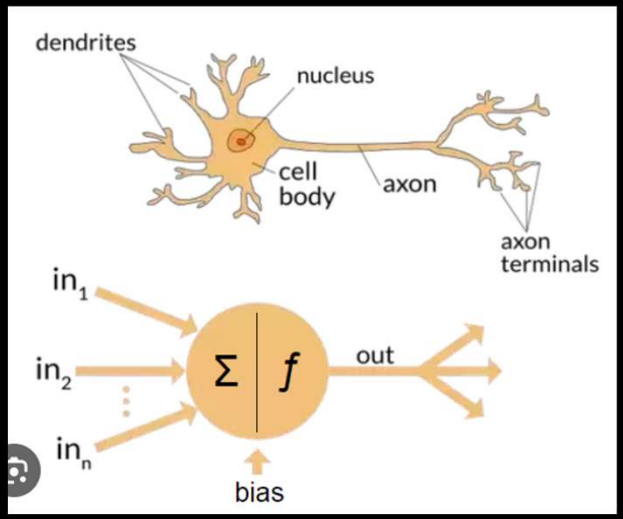
Deep Learning

Deep Learning

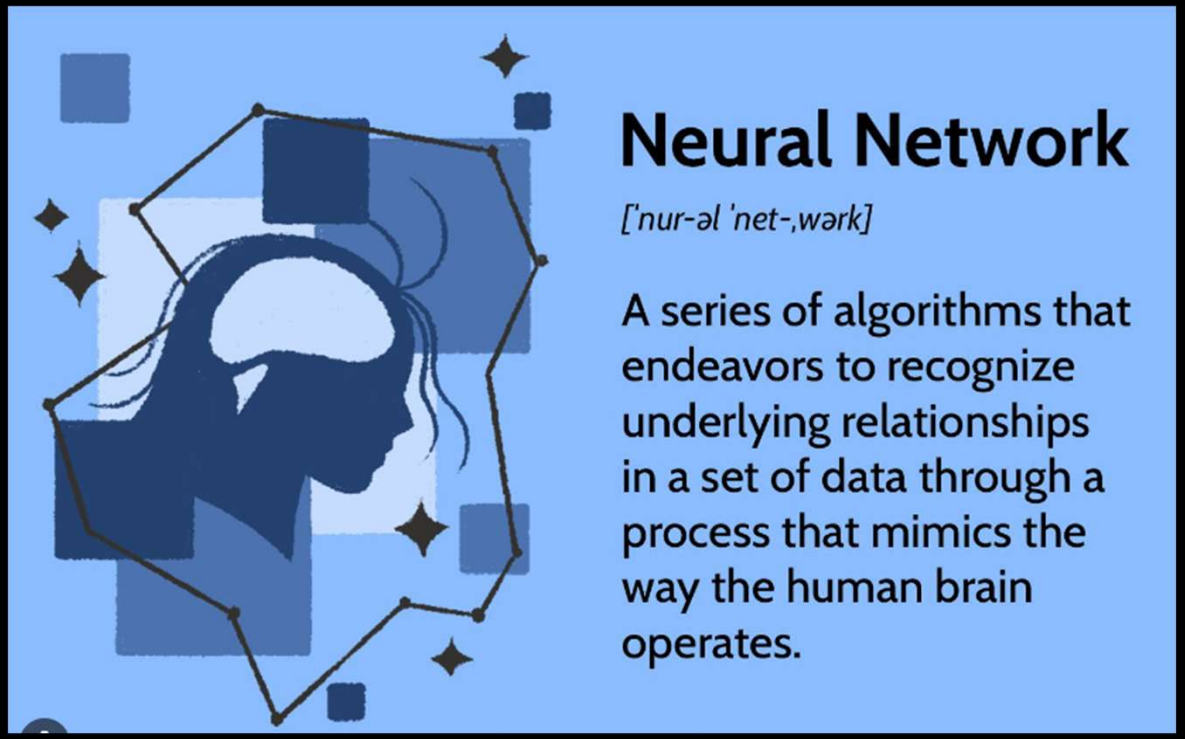


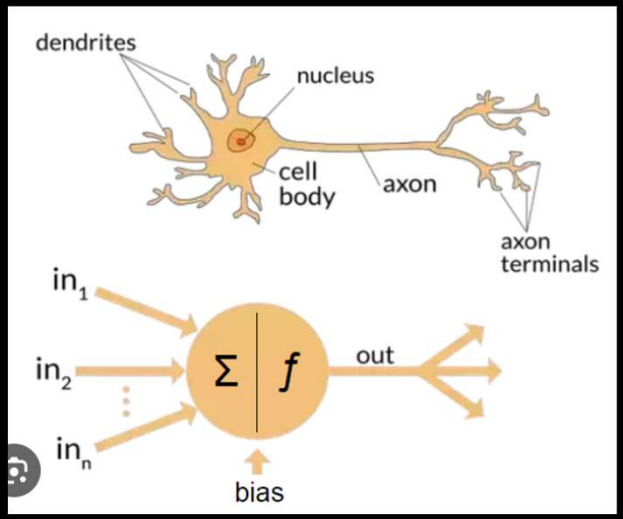
Neural Network



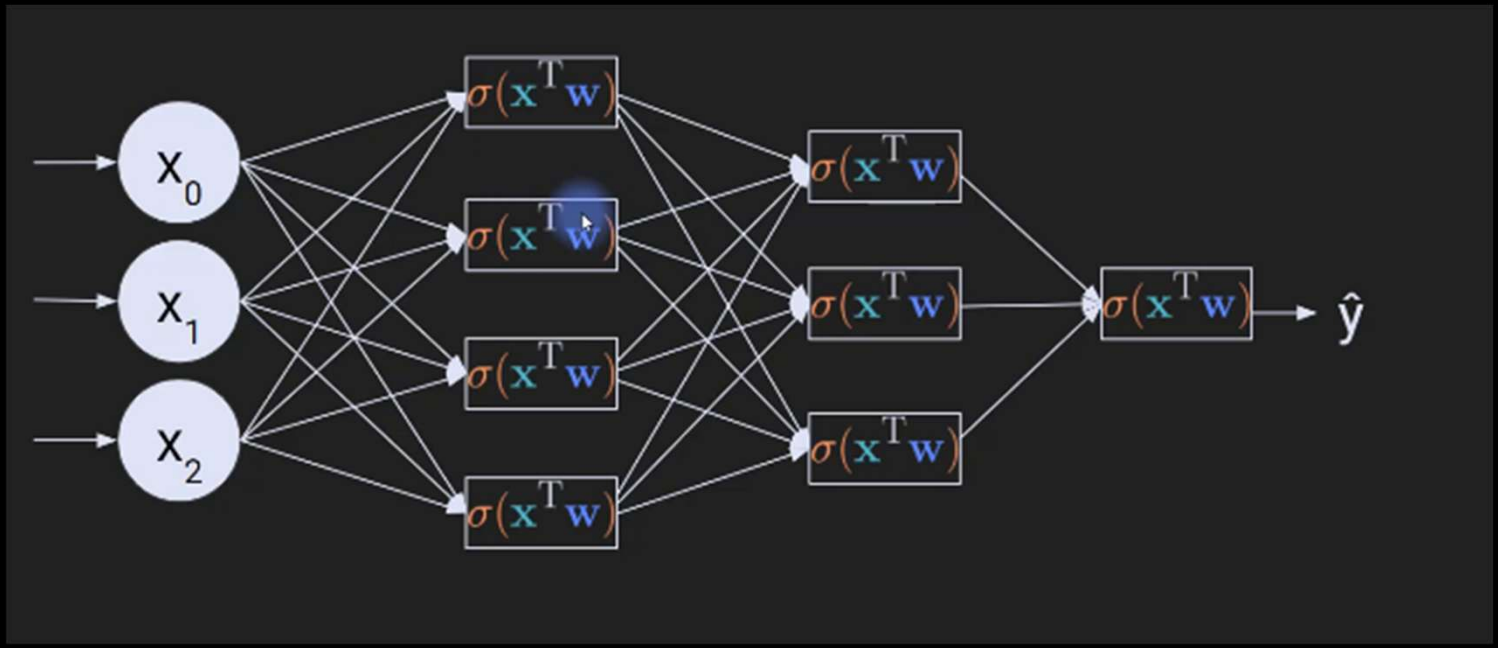


Neural Network



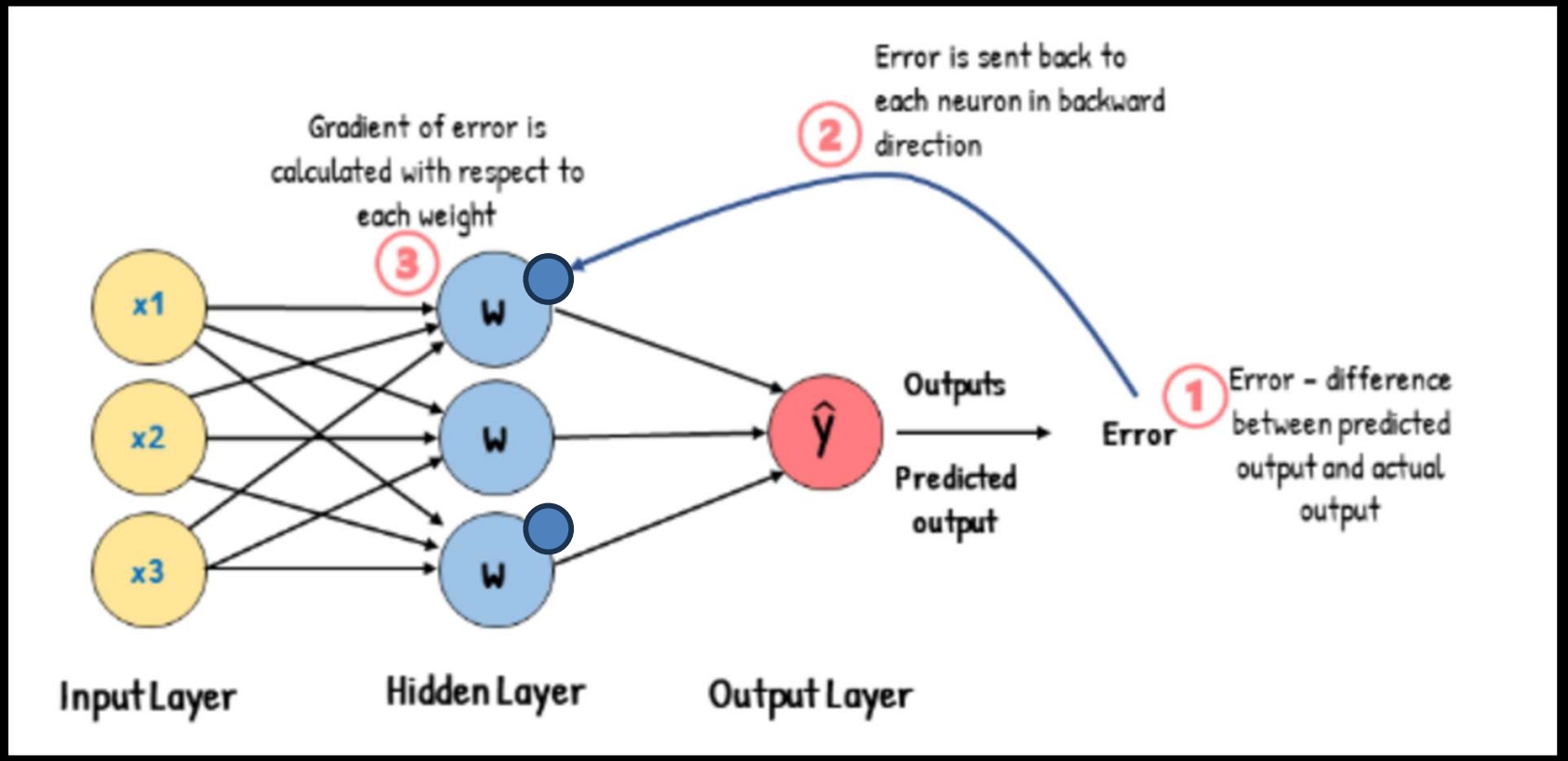


A fully connected neural network



Nodes, Layers, Weights

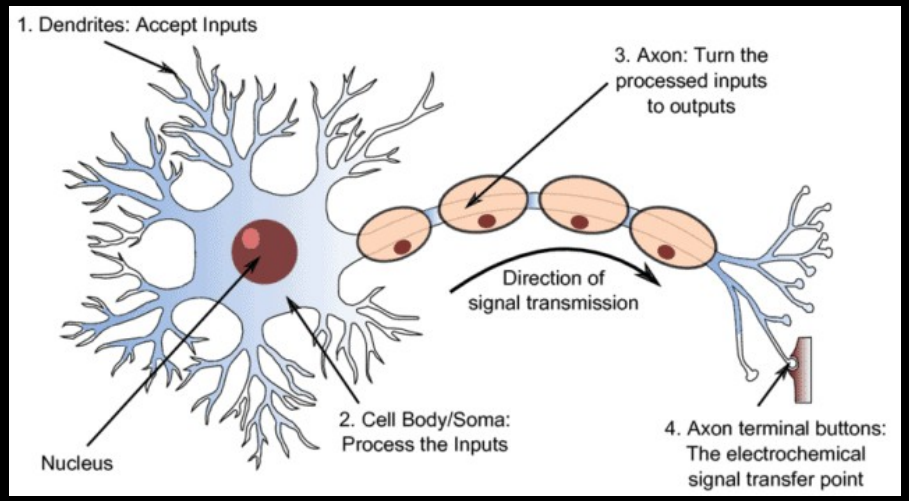
Neural Network



Nodes, Activation Functions, Layers, Weights and bias, Loss function, Gradient descent, back propagation

Neuron (or node) – building block of a neural network

- Inspired by biological brain cells
- Receives numerical information, perform mathematical operation, and pass the resulting number to the next layer
- **Inputs (x)**: The neuron receives data from the outside world or from neurons in the previous layer.
- **Weights (w)**: Every single incoming connection has its own weight. The neuron uses these weights to decide which inputs are the most important.
- **Bias (b)**: Each neuron typically has exactly one bias.
- **Parameters**: This is the collective total of the weights and the bias for that specific neuron.



Weights, biases and parameters

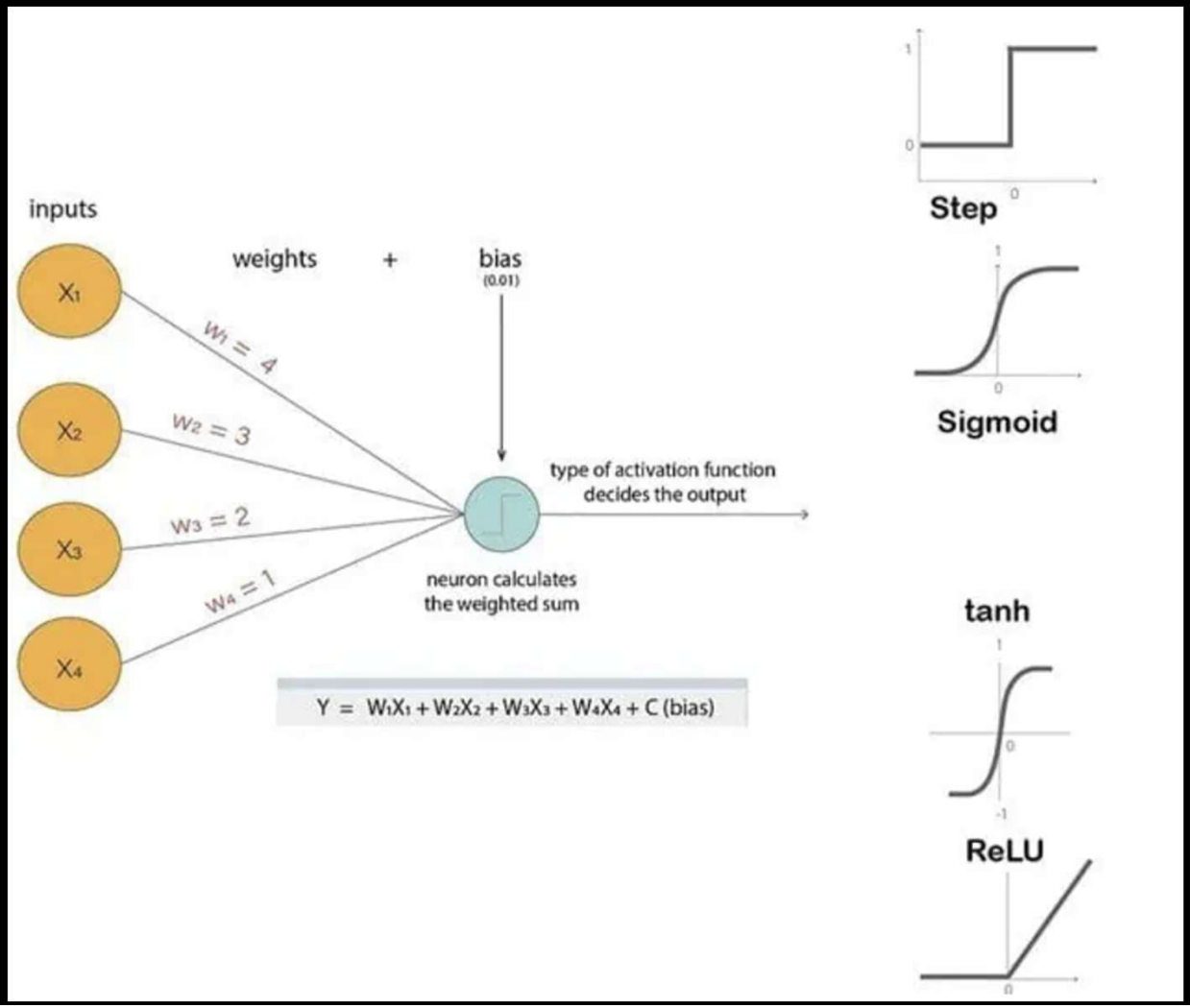
If a single neuron receives inputs from 5 different neurons in the previous layer, how many parameters does this single neuron have?

It has 5 weights (one for each incoming connection).
It has 1 bias (belonging to the neuron itself).
Therefore, this single neuron has 6 parameters total

Since weights constitute ~99% of trainable parameters, these terms are used interchangeably in the context of LLMs

Since each neuron is connected to every other neuron, for d=4096 means 4096*4096 weights between 2 layers, and just 4096 biases (~.024$)!

Weighted sum and activation

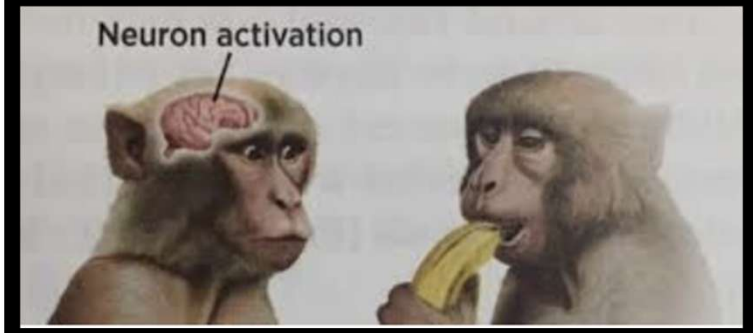


Step

Sigmoid

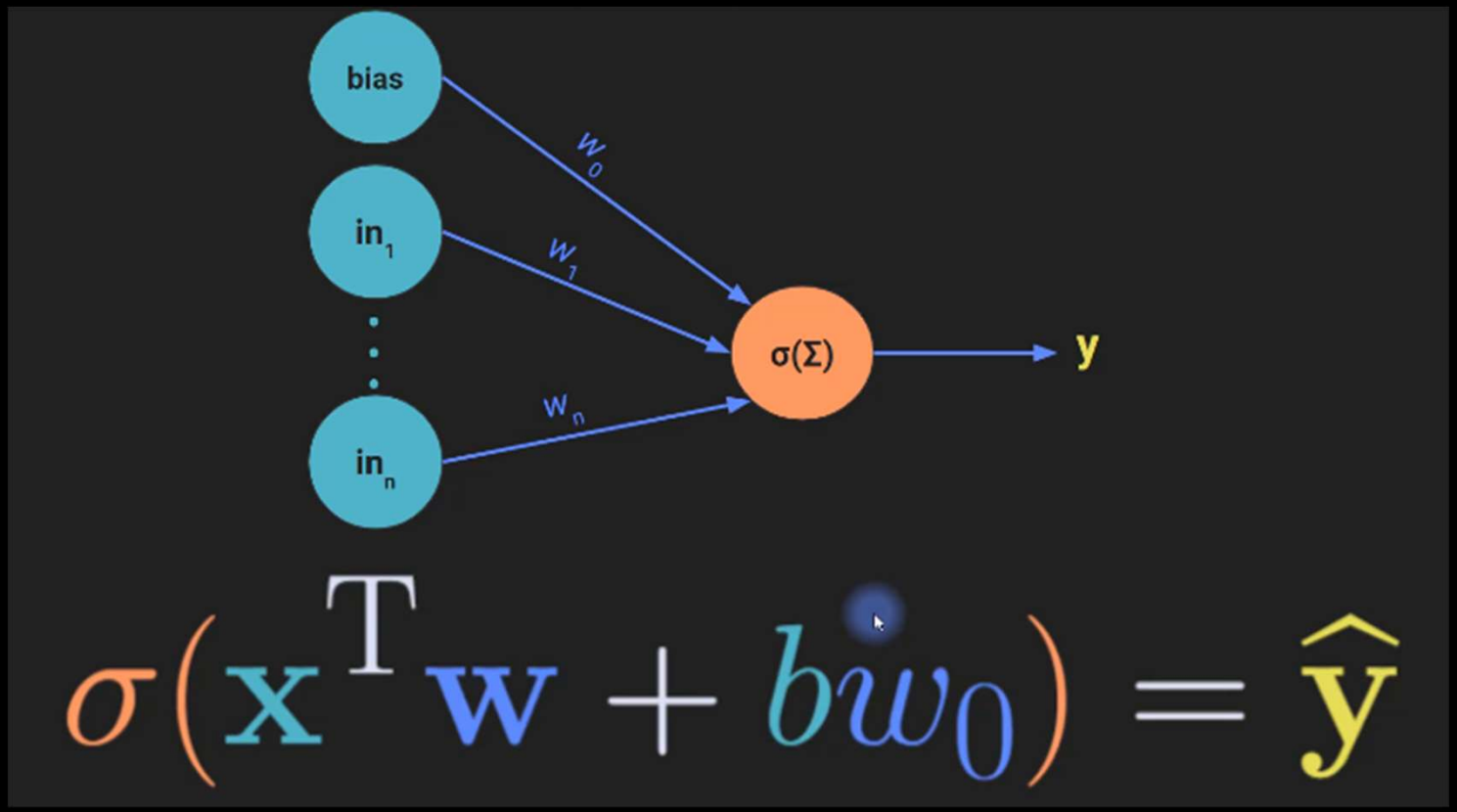
tanh

ReLU

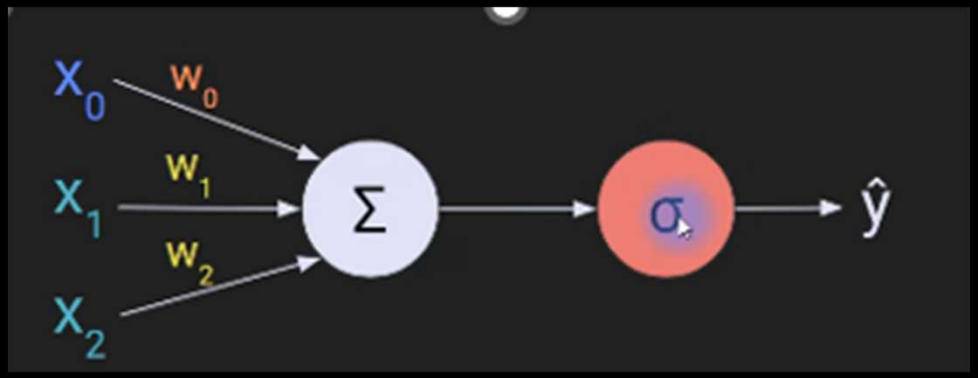


Step 1: Linear transformation

(Weighted sum of inputs + Bias)



Step 2: Non linear function



X0 = Bias

X1, X2: Features/Inputs

W0, W1, W2: Weights

Sigma: Linear weighted sum

Theta: Non Linear Activation function

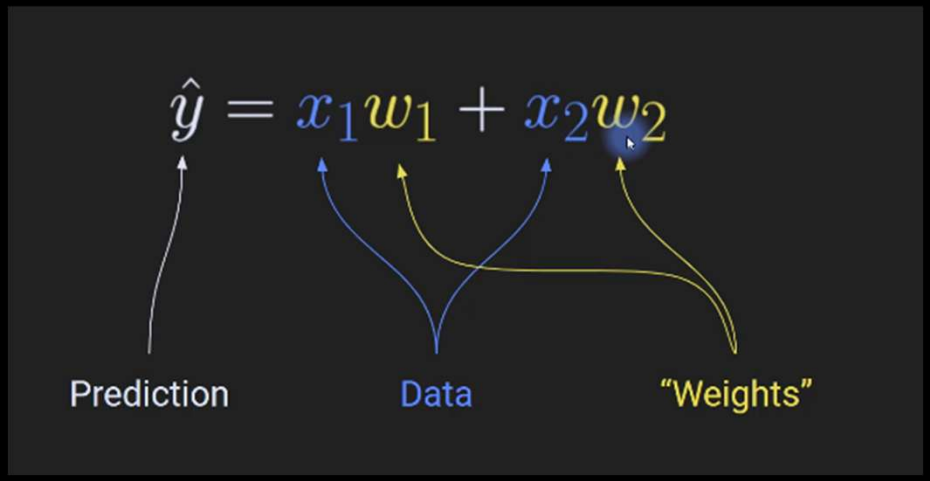
Y = output

Deep Learning excels in non linear solutioning

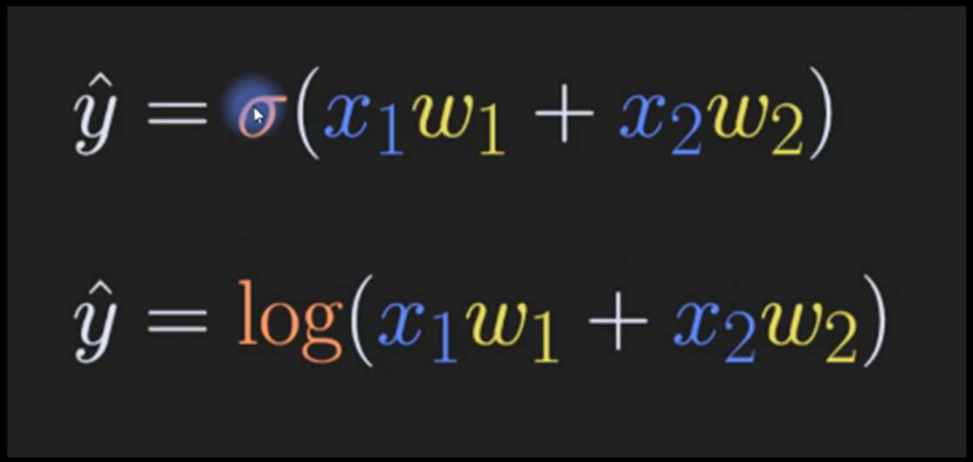
https://medium.com/@wysocki.przemyslaw/why-do-we-need-activation-functions-1874a53b1b15

Transformation and activation functions

Step 1: Linear transformation
Output: Weighted sum of inputs



Step 2: Activation function (non linear),
eg ReLU. Softmax or Sigmoid
Output: activated value



Model "Learns" the weights by a mechanism called Backpropagation

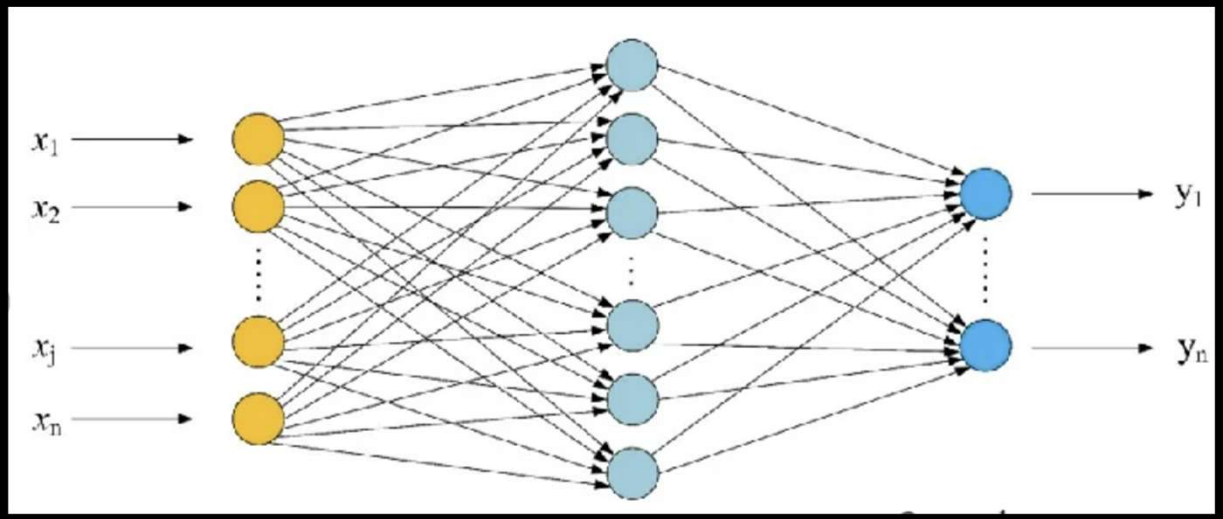
Fully Connected Neural Network (FCNN)

Key idea: **feature interaction**

Each neuron in the next layer can look at ALL inputs simultaneously

This allows it to learn:

- combinations
- interactions
- dependencies



Input Layer Hidden Layer Output Layer

Fully Connected Neural Network (FCNN) - intuition

Suppose you are predicting cardiac risk

Inputs:

Age
Blood Pressure
Cholesterol
Diabetes

- Without full connections

Neurons only see:

Age alone
BP alone

Cannot learn:
BP is dangerous only if Diabetes is present

- With full connections

Each neuron sees:

Age + BP + Cholesterol + Diabetes

So it can learn:

If BP × Diabetes is high → activate strongly
If young age → suppress risk”

Why are multiple neurons (width) needed?

Multiple neurons – width

Helps learn complex decision boundaries.

One neuron – may learn straight line/linear relation

But if data contains a pattern where points inside a circle are class1 and outside, class2

Combine three neurons, and you can draw a triangle.
Combine a hundred, and you can map out a highly complex, squiggly boundary that perfectly wraps around your data.

Why are multiple layers (depth) needed?

Multiple layers – depth

Helps with hierarchical learning

Layers allow the network to break the problem down into a sequence of increasingly complex steps.
The output of one layer becomes the input for the next, allowing the network to learn abstract concepts from raw data.

For example: Layer 1: lines Layer 2: shapes Layer 3: face, hands Layer 4: specific person

How is the increasing complexity achieved? Do we configure which layer should look at what features?

This is automatic! Early layers only "see" small, basic inputs.
Later layers aggregate data from multiple early neurons, allowing them to piece together high-level features.

Walkthrough – diabetes onset prediction

Dataset

Feature	Type
BP	numeric
Sugar	numeric
Age	numeric
Label	diabetes onset (0/1)

👉 Total:

- **3 input features**
- **1 binary output**

Forward pass - Layers

For tabular dataset, typically #neurons in input layer = # features

Input Layer: 3 neurons
Neuron 1 → BP
Neuron 2 → Sugar
Neuron 3 → Age
Input data: [BP, Sugar, Age] = [140,180,55]
Output of input layer will be the same x=[140,180,55]

Relationship is non-linear
Example: High sugar alone → maybe fine
High sugar + high BP → risky

Hence we need hidden layers

```
Input (3) → Hidden Layer (8) → Hidden Layer (4) → Output (1)
```

Forward pass - Hidden Layer1

Neurons: 8

Each neuron computes:

$$z = w_1 \cdot BP + w_2 \cdot Sugar + w_3 \cdot Age + b$$

Then applies activation:
ReLU

What these neurons learn (intuition)

Each neuron becomes a **feature detector**:
Neuron 1 → “high sugar pattern”
Neuron 2 → “BP × Age interaction”
Neuron 3 → “young + high sugar”
etc.

Forward pass → **Loss calculation** → Backpropagation → Optimizer step → Repeat

Forward pass - Hidden Layer *n*

Hidden Layer 2

Neurons: 4

Learn higher-level patterns like:
“metabolic risk”
“age-adjusted risk”

Forward pass → **Loss calculation** → Backpropagation → Optimizer step → Repeat

Forward pass – output layer

Binary classification:
Neurons: 1

Multi-class classification:
Neurons = number of classes
Each neuron will predict the ***probability*** of that class

Regression
only one neuron – predicts the number

Forward pass → **Loss calculation** → Backpropagation → Optimizer step → Repeat

Loss function – calculates error

The loss function compares the predicted output with the actual label.

```
Loss = difference between actual y and predicted ŷ
```

For classification, common loss is:
`Binary Cross Entropy`

For regression, common loss is:
`Mean Squared Error`

Example:

```
Actual label = 1
Predicted probability = 0.82
Loss = small

Actual label = 1
Predicted probability = 0.15
Loss = large
```

Forward pass → **Loss calculation** → Backpropagation → Optimizer step → Repeat

Backpropagation – calculates gradients

Backpropagation does **not update weights directly**.
It calculates:
`How much each weight contributed to the loss`

This is called the **gradient**. For every weight, it computes:
`∂Loss / ∂Weight`

Meaning:
`If this weight changes slightly, how much will the loss change?`

After backpropagation, each weight has a gradient attached to it.
Example:
```
Weight w1 = 0.40
Gradient for w1 = +0.25
```
This means increasing `w1` increases the loss, so the optimizer should reduce it.

Forward pass → **Loss calculation** → Backpropagation → Optimizer step → Repeat

Optimizer – update weights

The optimizer uses the gradients calculated by backpropagation to change the weights.

For simple gradient descent:

```
new weight = old weight - learning rate × gradient
```

Example:

```
old weight = 0.40
learning rate = 0.01
gradient = 0.25

new weight = 0.40 - 0.01 × 0.25
new weight = 0.3975
```

Backpropagation calculates gradients.
Optimizer updates weights.

```
zero_grad()      → clear old gradients
loss.backward() → backpropagation: calculate new gradients
step()           → optimizer updates weights
```

Forward pass → **Loss calculation** → Backpropagation → Optimizer step → Repeat

Summary

```
Model makes prediction
↓
Loss measures mistake
↓
Backpropagation finds who caused the mistake
↓
Optimizer decides how to punish/correct each weight
↓
Weights are updated
↓
Next batch starts
```

Analogy

```
Loss function = exam score
Backpropagation = finding which topics caused marks to be lost
Optimizer = deciding how much study effort to shift to each topic
Weight update = actually changing the preparation strategy
```

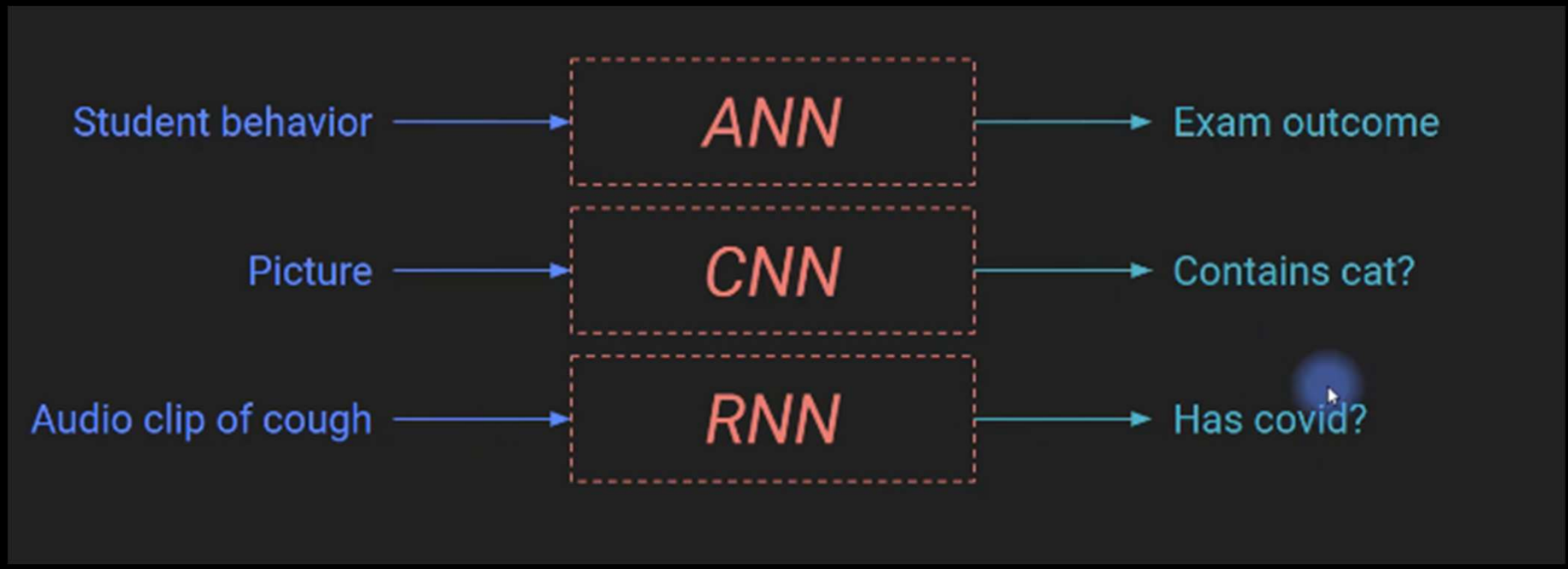
How does neural network do unsupervised learning?

Method	What is the loss?
Autoencoder	Difference between input and reconstructed input
Contrastive learning	Similar examples close, different examples far
Deep clustering	Distance from cluster centers
Masked prediction	Predict missing parts of input
Anomaly detection	Reconstruction error or density-based objective

An autoencoder is one of the most common neural-network-based unsupervised learning methods.
It tries to reconstruct the input.

```
Input data → Encoder → compressed representation → Decoder → reconstructed input
```

ANN, CNN, RNN

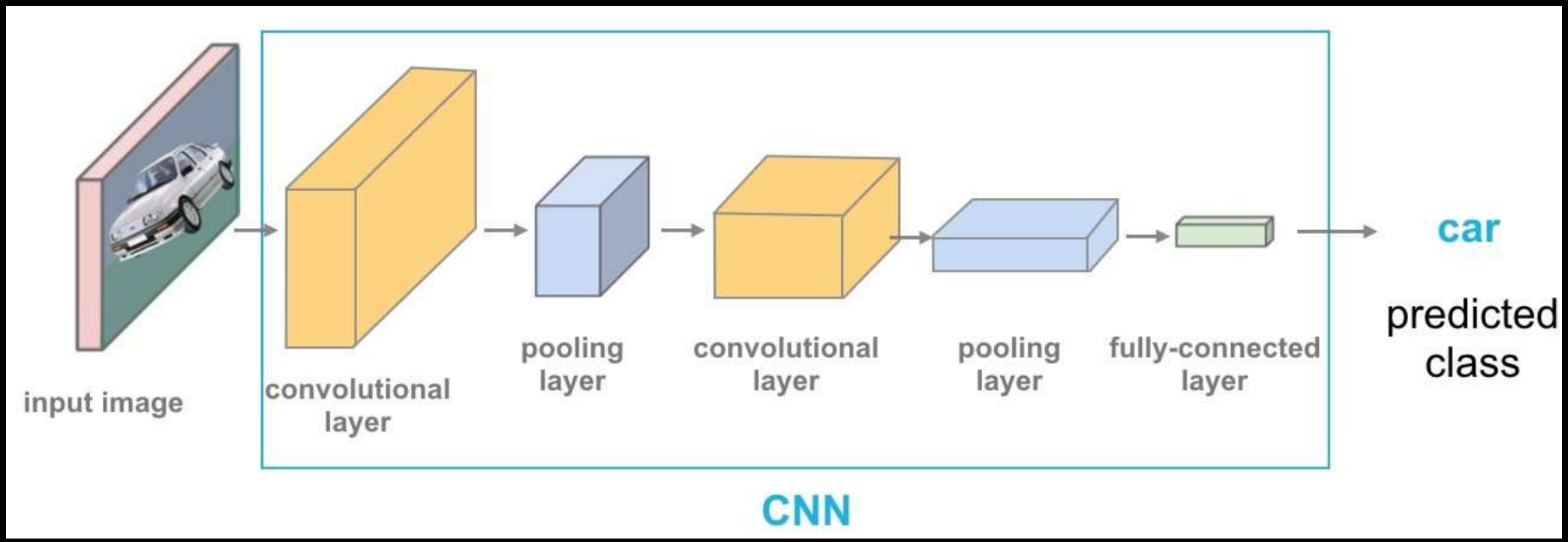


ANN - Artificial Neural Network

CNN – Convolutional Neural Network (typically used for Image input)

RNN – Recursive Neural Network (typically used for timeseries input)

Convolutional Neural Network (CNN)



Convolution layer: Detects features such as edges, shapes resulting in a feature map

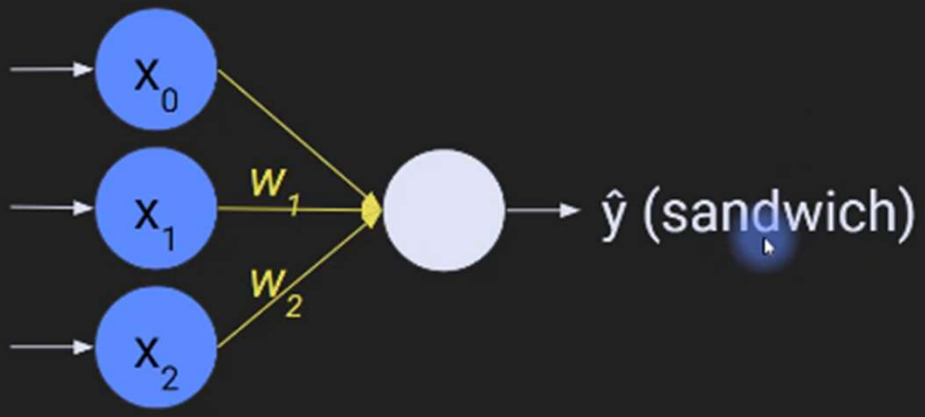
Max Pooling Layer: Reduces the size of the feature map, focusing only on prominent features

Flattening: Convert the 2D pooled feature map into a 1D vector

Dense Layers: Combines the features to make a prediction

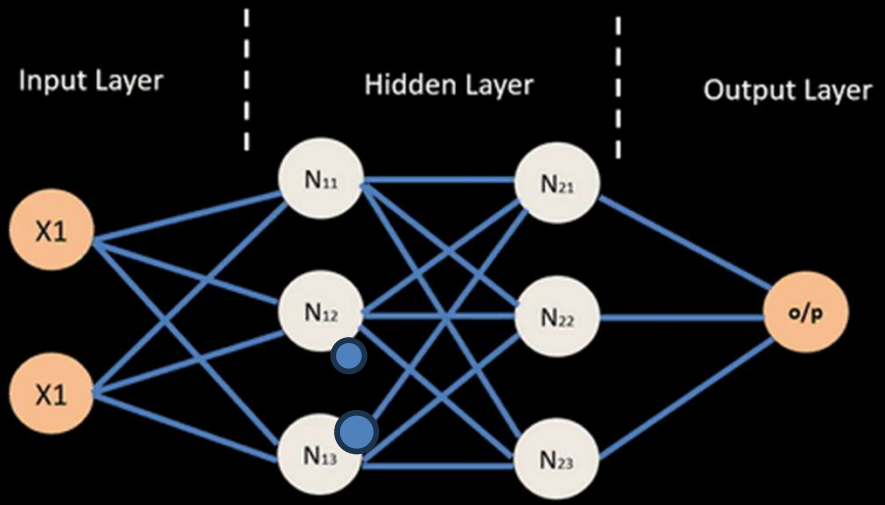
Softmax Output layer: Outputs the possibilities of each digit class (0-9)

Deep Learning



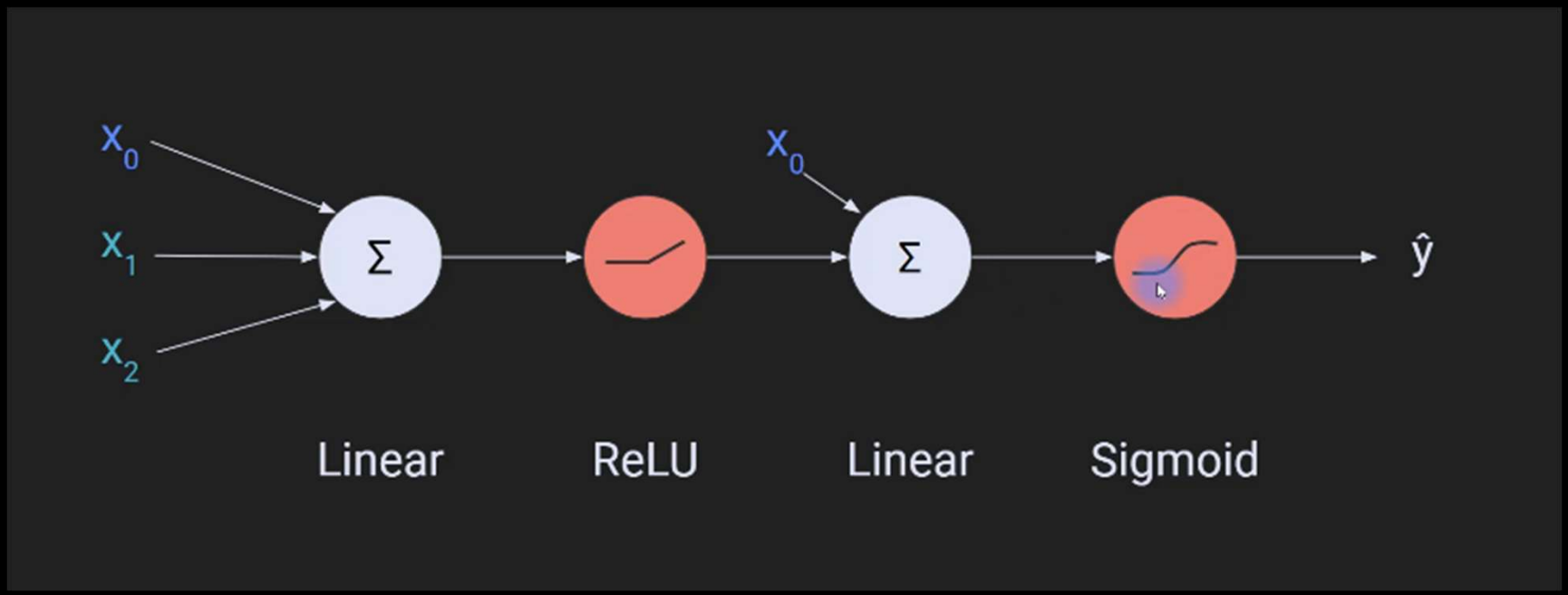
Backward propagation
Adjust weights and biases based on error in output

Backward propagation



Backward propagation
Adjust weights and biases based on error in output

Multi layer flow



Sigmoid: typically used as activation function for output layer for binary classification

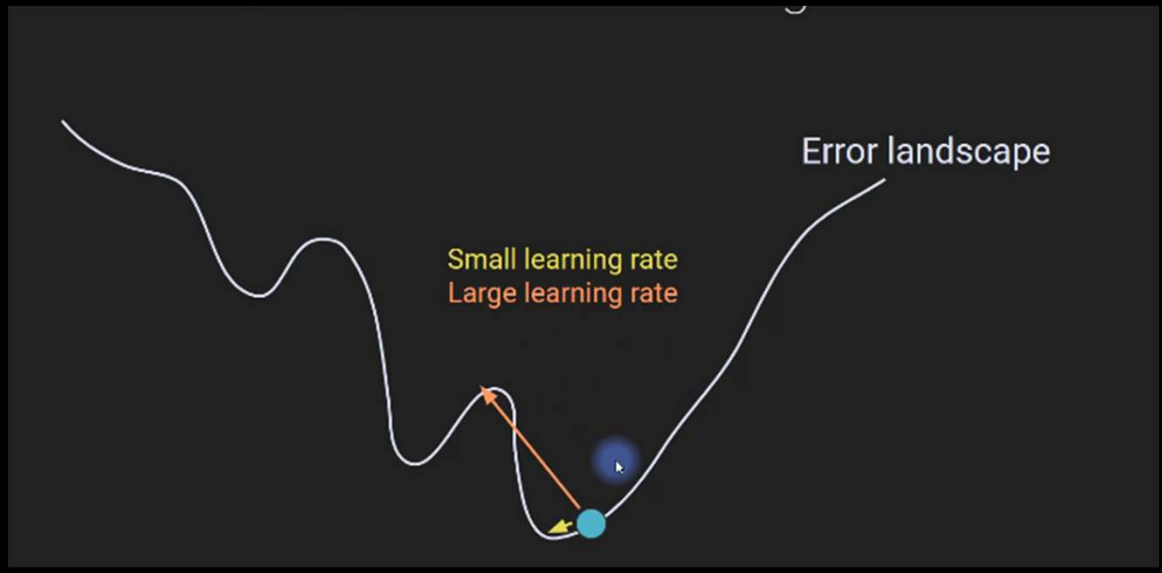
Learning rate

The **learning rate** in a neural network is a hyperparameter that controls how much the model's weights are adjusted during each step of the training process.

It determines the **step size** at each iteration while moving toward the optimal weights that minimize the model’s loss function.

Too high learning rate can lead to overshooting.

Too low learning rate can lead to slow convergence

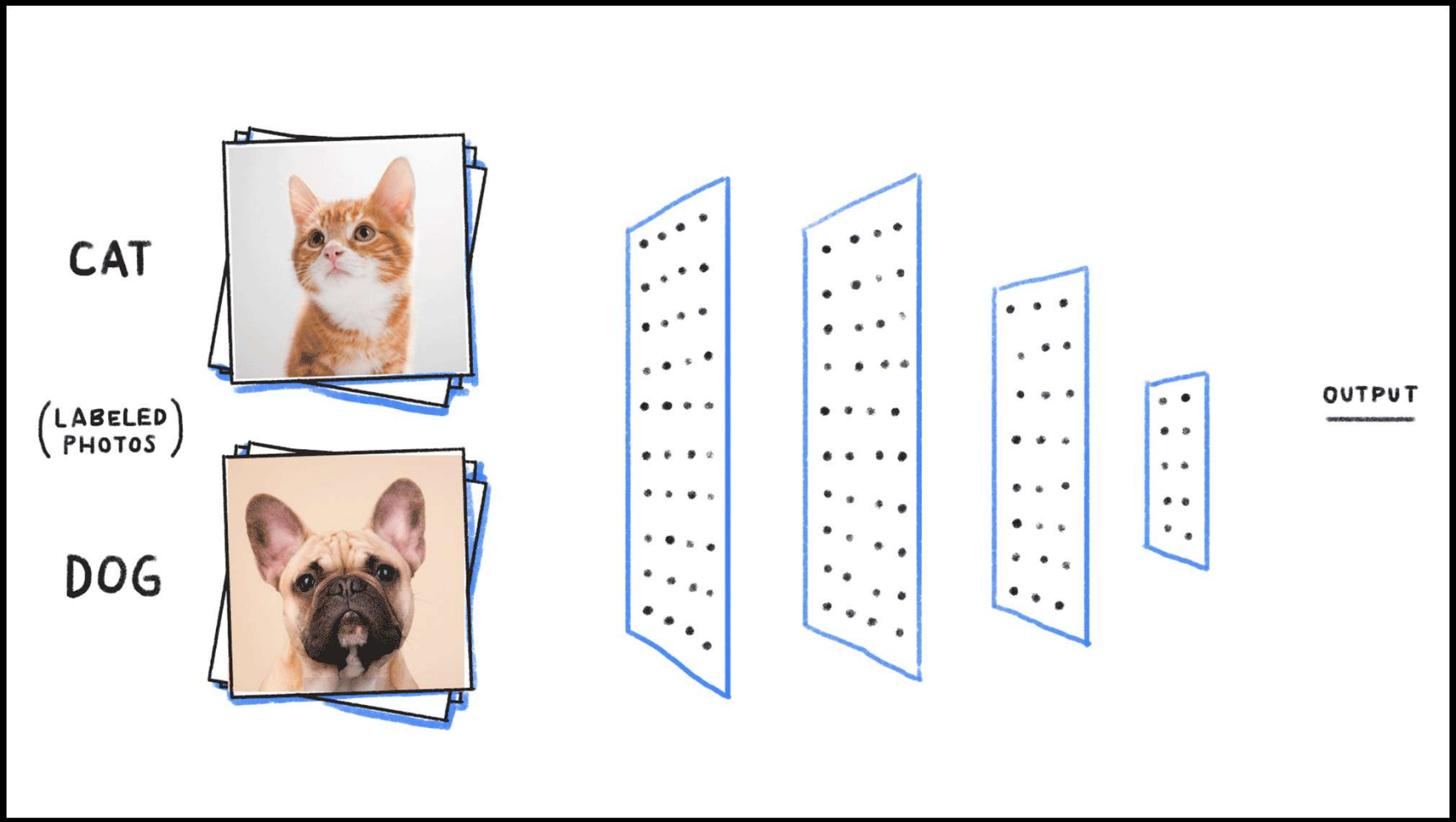


Demo

Machine Learning Playground (ml-playground.com)

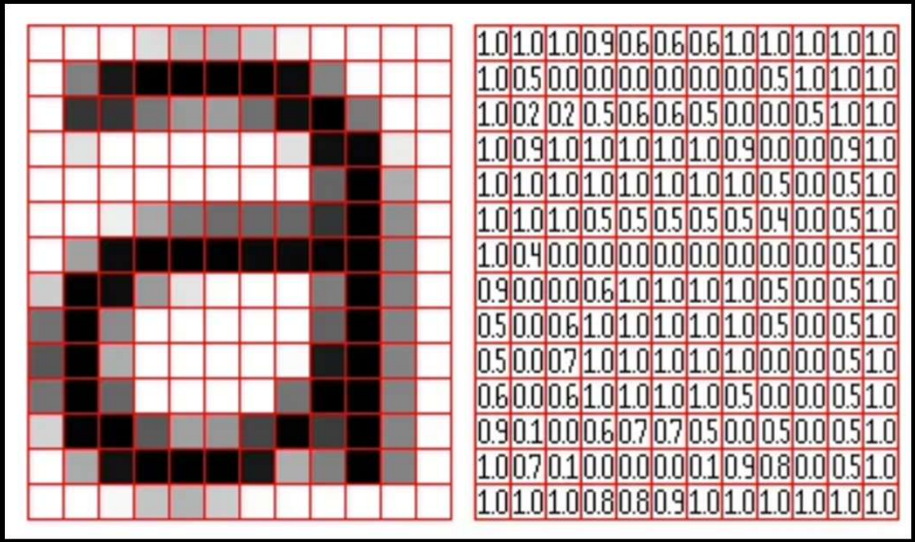
https://playground.tensorflow.org/

Convolutional Neural Network (CNN)



The first step – breaking down image, text into machine understandable format

Image is represented as matrix of numbers

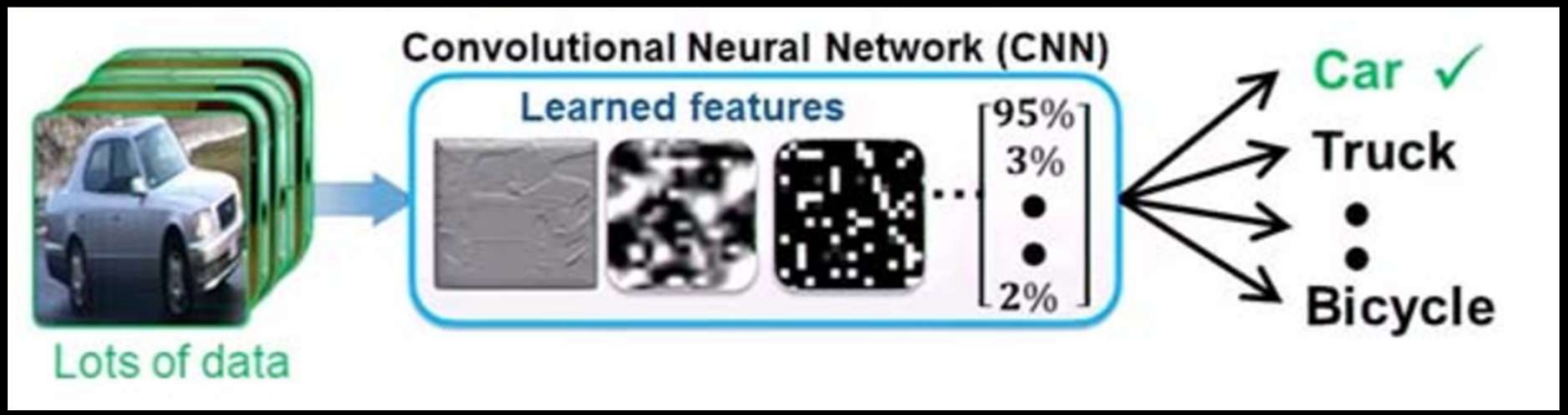


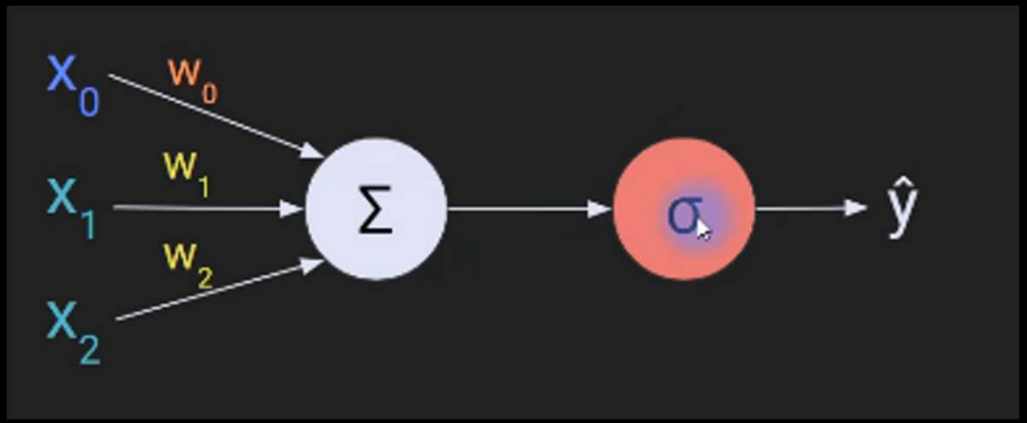
1.0	1.0	1.0	0.9	0.6	0.6	0.6	1.0	1.0	1.0	1.0	1.0
1.0	0.5	0.0	0.0	0.0	0.0	0.0	0.0	0.5	1.0	1.0	1.0
1.0	0.2	0.2	0.5	0.6	0.6	0.5	0.0	0.0	0.5	1.0	1.0
1.0	0.9	1.0	1.0	1.0	1.0	1.0	0.9	0.0	0.0	0.9	1.0
1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	0.5	0.0	0.5	1.0
1.0	1.0	1.0	0.5	0.5	0.5	0.5	0.5	0.4	0.0	0.5	1.0
1.0	0.4	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.5	1.0
0.9	0.0	0.0	0.6	1.0	1.0	1.0	1.0	0.5	0.0	0.5	1.0
0.5	0.0	0.6	1.0	1.0	1.0	1.0	1.0	0.5	0.0	0.5	1.0
0.5	0.0	0.7	1.0	1.0	1.0	1.0	1.0	0.0	0.0	0.5	1.0
0.6	0.0	0.6	1.0	1.0	1.0	1.0	0.5	0.0	0.0	0.5	1.0
0.9	0.1	0.0	0.6	0.7	0.7	0.5	0.0	0.5	0.0	0.5	1.0
1.0	0.7	0.1	0.0	0.0	0.0	0.1	0.9	0.8	0.0	0.5	1.0
1.0	1.0	1.0	0.8	0.8	0.9	1.0	1.0	1.0	1.0	1.0	1.0

Text is represented as sequence of numbers, called vectors

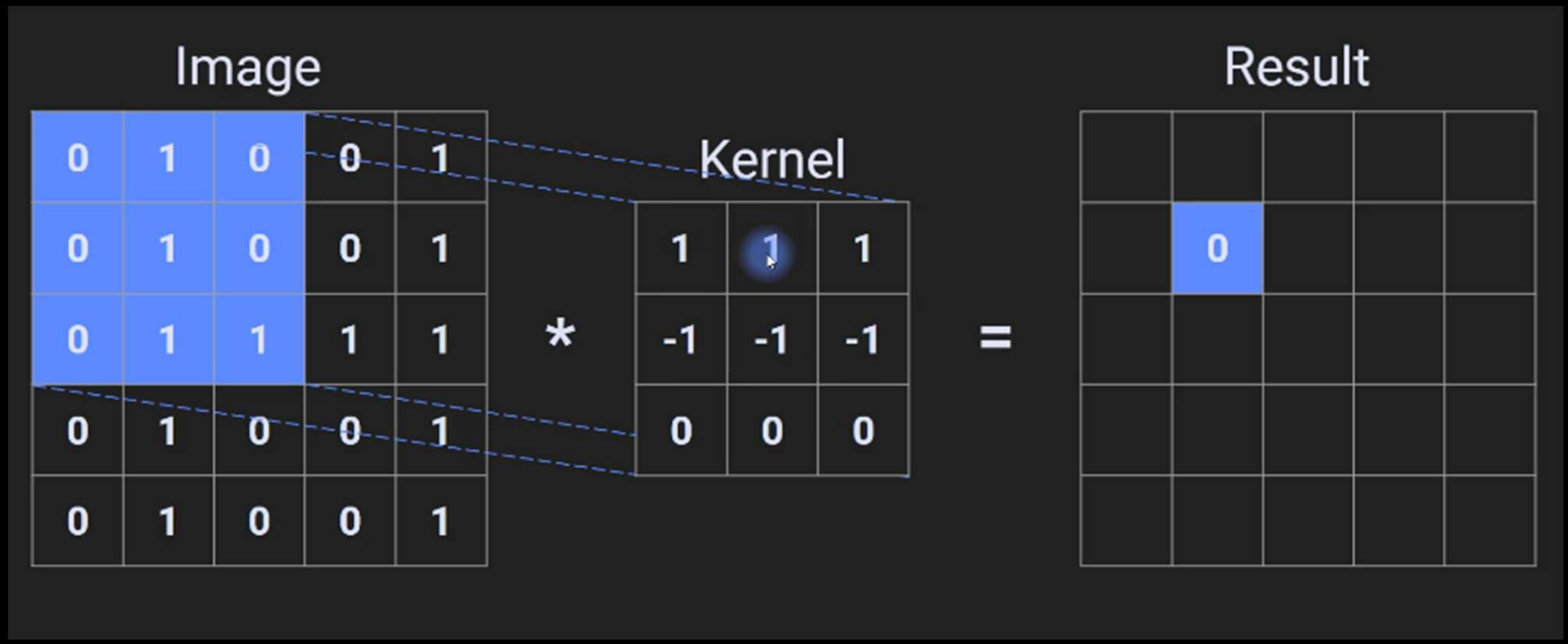
word	vector
The	(0.12, 0.23, 0.56)
Cardinals	(0.24, 0.65, 0.72)
will	(0.38, 0.42, 0.12)
win	(0.57, 0.01, 0.02)
the	(0.53, 0.68, 0.91)
world	(0.11, 0.27, 0.45)
series	(0.01, 0.05, 0.62)

The second step – train the neural network with lots of data



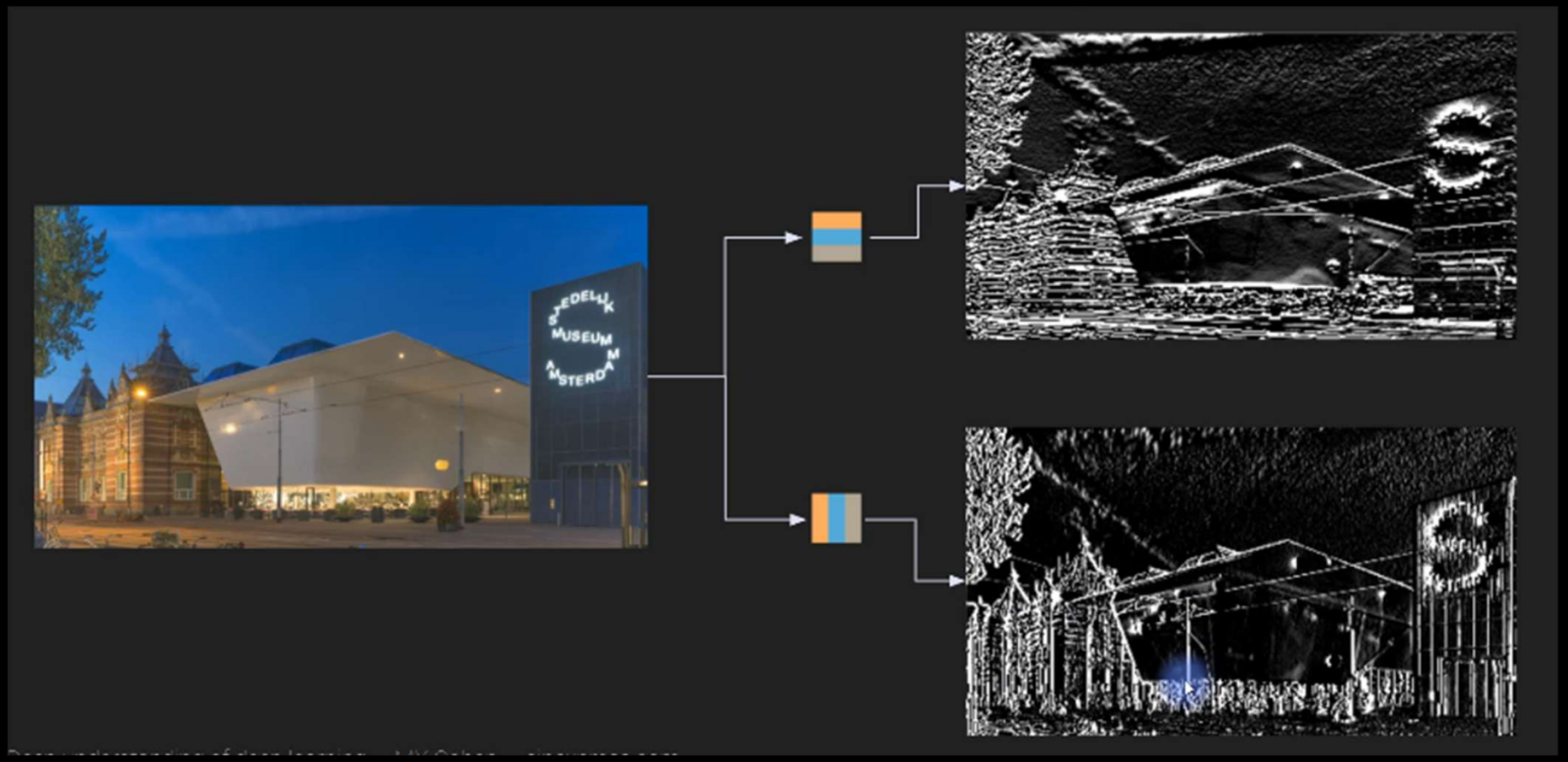


Convolution



Identifies the features in the data, for example, in an image it identifies the edges, textures, pattterns

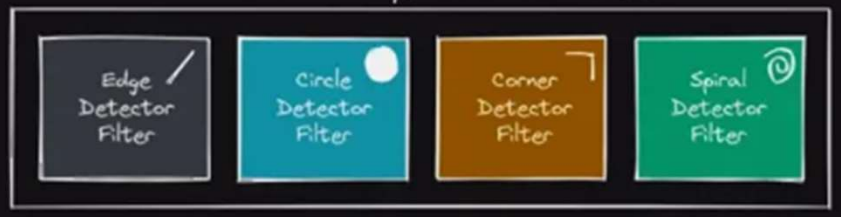
Convolution



Horizontal and vetical kernels applied to an image – output is a feature map

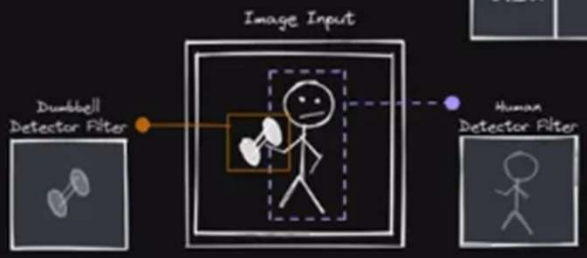
CONVOLUTIONAL LAYERS & FILTERS

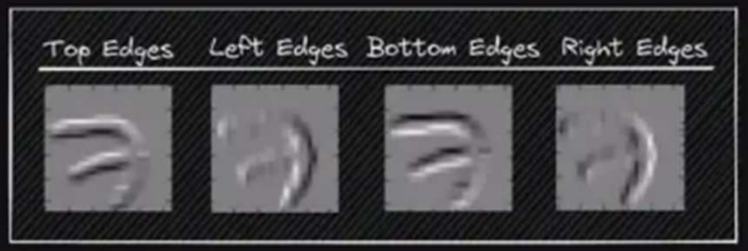
Convolutional Layer with Four Filters



3 x 3 Convolutional Filter

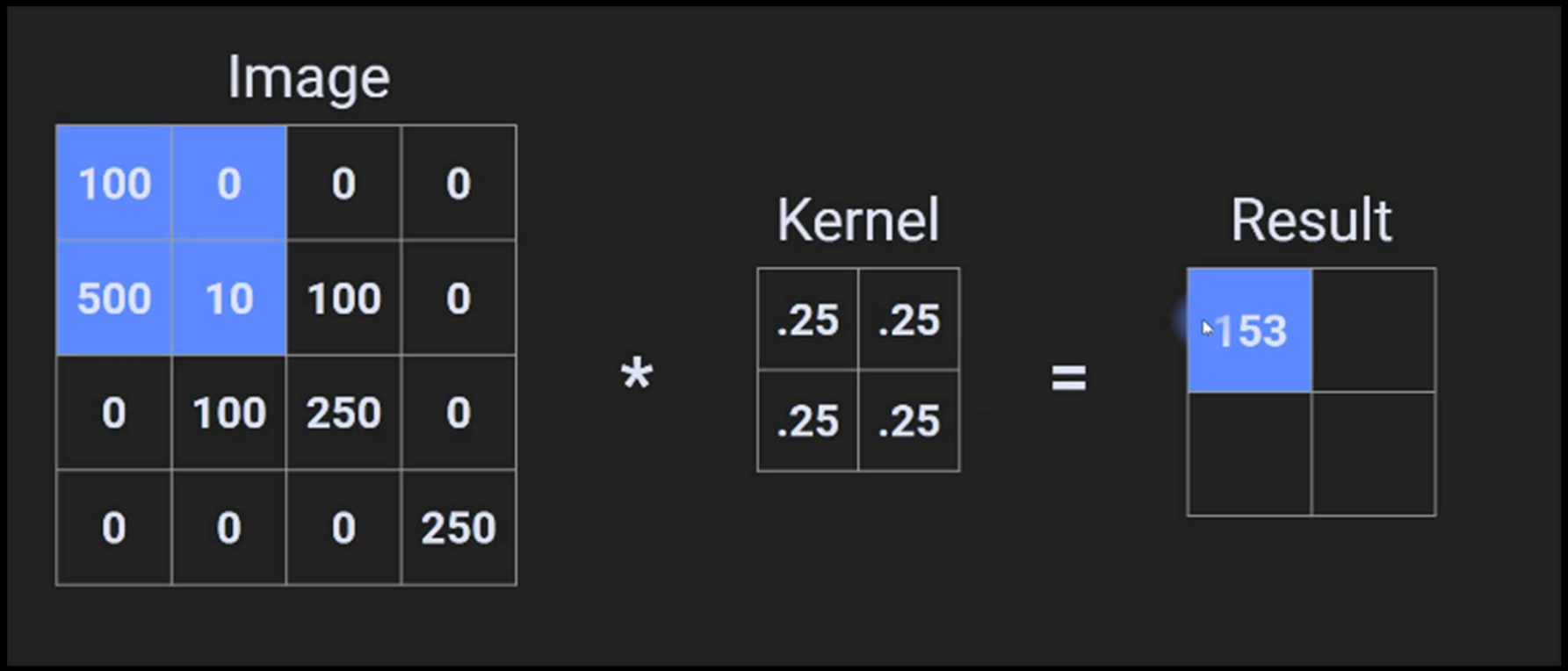
0.1	-0.92	0.5
1.4	0.39	-1.1
-0.32	-1.2	0.84





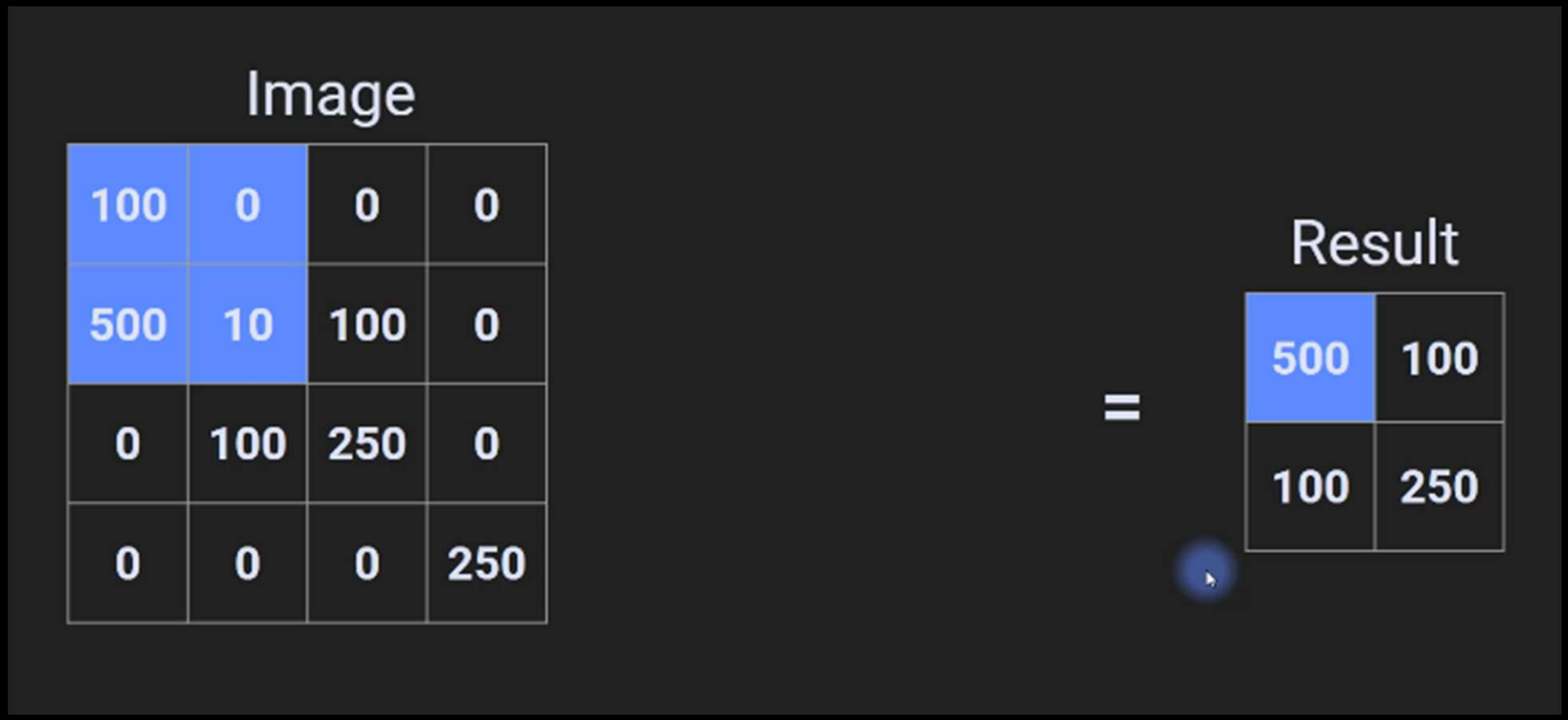
90%

Pooling



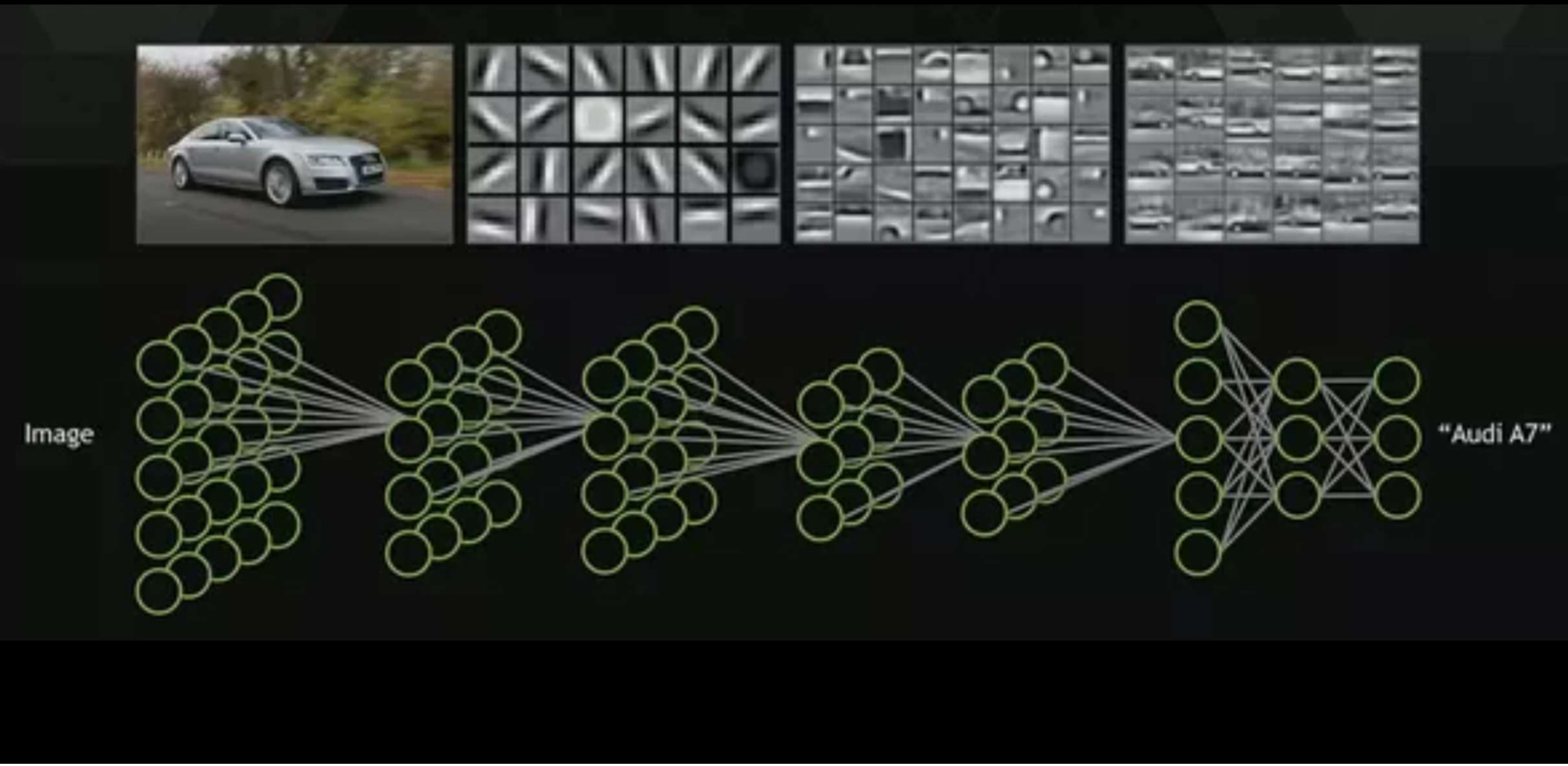
Purpose is to reduce dimensionality
Above is doing pooling with a kernel 2x2 with a stride of 2

Pooling

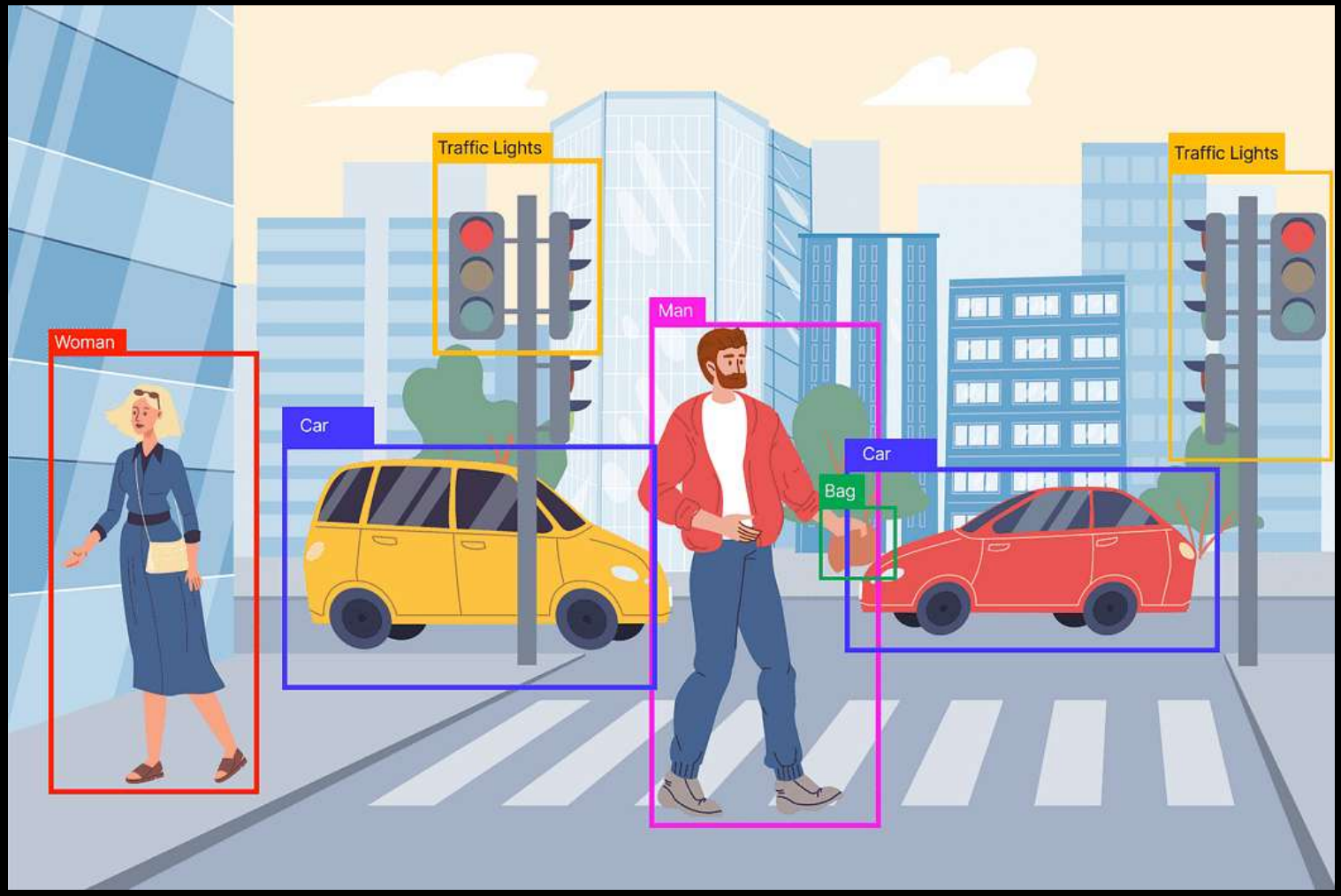


Max pooling

Pooling



Object Detection



CNN advancements for object detection

Can CNN natively do it?

Advancements to CNN to make object detection possible”

Faster R-CNN (Region Proposal Network RPN)
Two-Stage Detection: Faster R-CNN works in two stages:
1. A Region Proposal Network (RPN) suggests possible object locations.
2. The classification head refines these proposals and assigns class labels.

SSD (Single Shot MultiBox Detector)
SSD predicts objects from multiple feature maps, each tuned to detect different object sizes. Early layers find small objects, and deeper layers detect larger ones

YOLO (You Look Only Once)
YOLO processes the image in a single forward pass, dividing the image into a grid and predicting bounding boxes and class probabilities simultaneously.

How does YOLO work?

Input Image:

The input image is resized to a fixed size, usually 416x416 or 640x640.

Grid Division:

YOLO divides the image into an S x S grid (e.g., 13x13 for YOLOv3). Each grid cell is responsible for detecting objects whose center falls within the cell.

Bounding Box Prediction:

Each grid cell predicts B bounding boxes (e.g., 3 per cell in YOLOv3). For each bounding box, YOLO predicts center coordinates relative to the grid cell (x, y); Width and height relative to the image size (w, h), confidence score. Additionally, it predicts class probabilities for each cell.

Output:

The final output is a list of detected objects with their class labels, confidence scores, and bounding boxes.

How does training data for YOLO look like?

1 Image

The image itself (e.g., `image1.jpg`).

2 YOLO Annotation File (`image1.txt`)

For each object in the image, the annotation file includes:

```php-template
<class_id> <x_center> <y_center> <width> <height>
```

Where:

- `class_id:` Index of the object class (e.g., 0 for "car", 1 for "person").
- `x_center:` Horizontal center of the bounding box (normalized: 0 to 1).
- `y_center:` Vertical center of the bounding box (normalized: 0 to 1).
- `width:` Width of the bounding box (normalized by image width).
- `height:` Height of the bounding box (normalized by image height).

Yolo versions

YOLOv1 (2016): Created by **Joseph Redmon**. First real-time object detection model.

YOLOv2 and YOLOv3 (2017–2018): Also developed by Redmon with improvements in accuracy and multi-scale detection.

YOLOv4 (2020): Released by Alexey Bochkovskiy after Redmon stopped working on computer vision due to ethical concerns.

YOLOv5 (2020): Released by Ultralytics, though not officially affiliated with the original creators.

YOLOv6, YOLOv7 (2022): Developed by different research groups, focusing on speed and efficiency.

YOLOv8 (2023): Released by Ultralytics with further performance improvements.

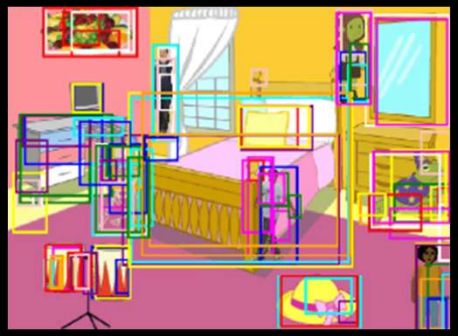
How to achieve fine grained detection?

Detecting hair line fracture? Detecting tumour in x-ray/ct-scan/mri scan images

- Adapt or extend YOLO
  increase input resolution, use custom anchors, domain specific pre-processing (equalization, windowing, de-noising)
  targeted augmentations such as small rotations
- Use Faster R-CNN
- Hyper parameter tuning, transfer learning, domain specific pre-trained models
- Advanced models such as FPN (Feature Pyramid Network) or PAN (Path Aggregation Network) can help detect objects at multiple scales.

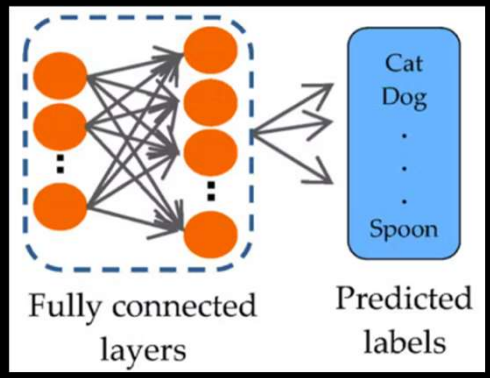
Labeling the detected object – transfer learning

Original Image

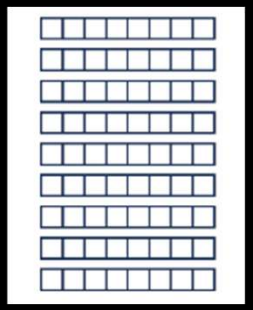


Object detection

Pre-trained models (ResNet, CLIP, ViT) with object labels

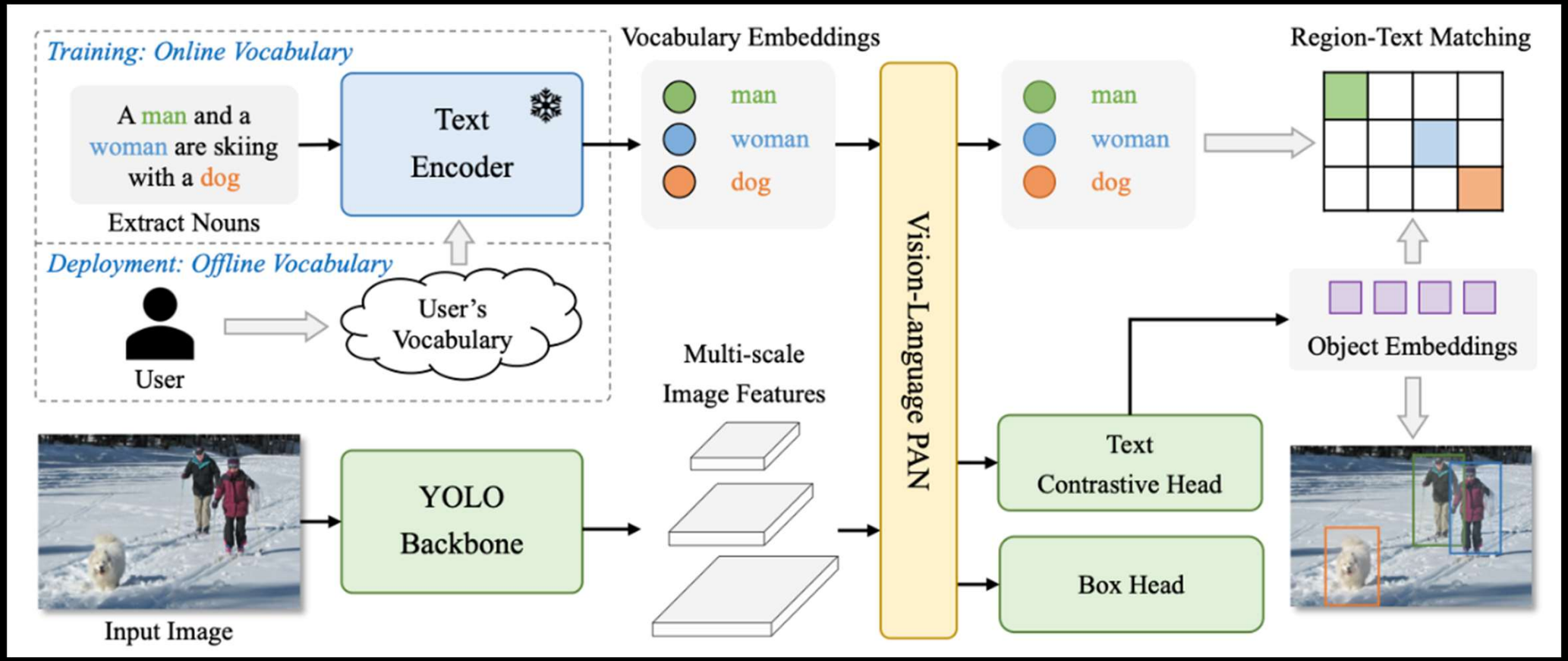


Transfer Learning



Object embeddings

Labeling the detected object – CLIP



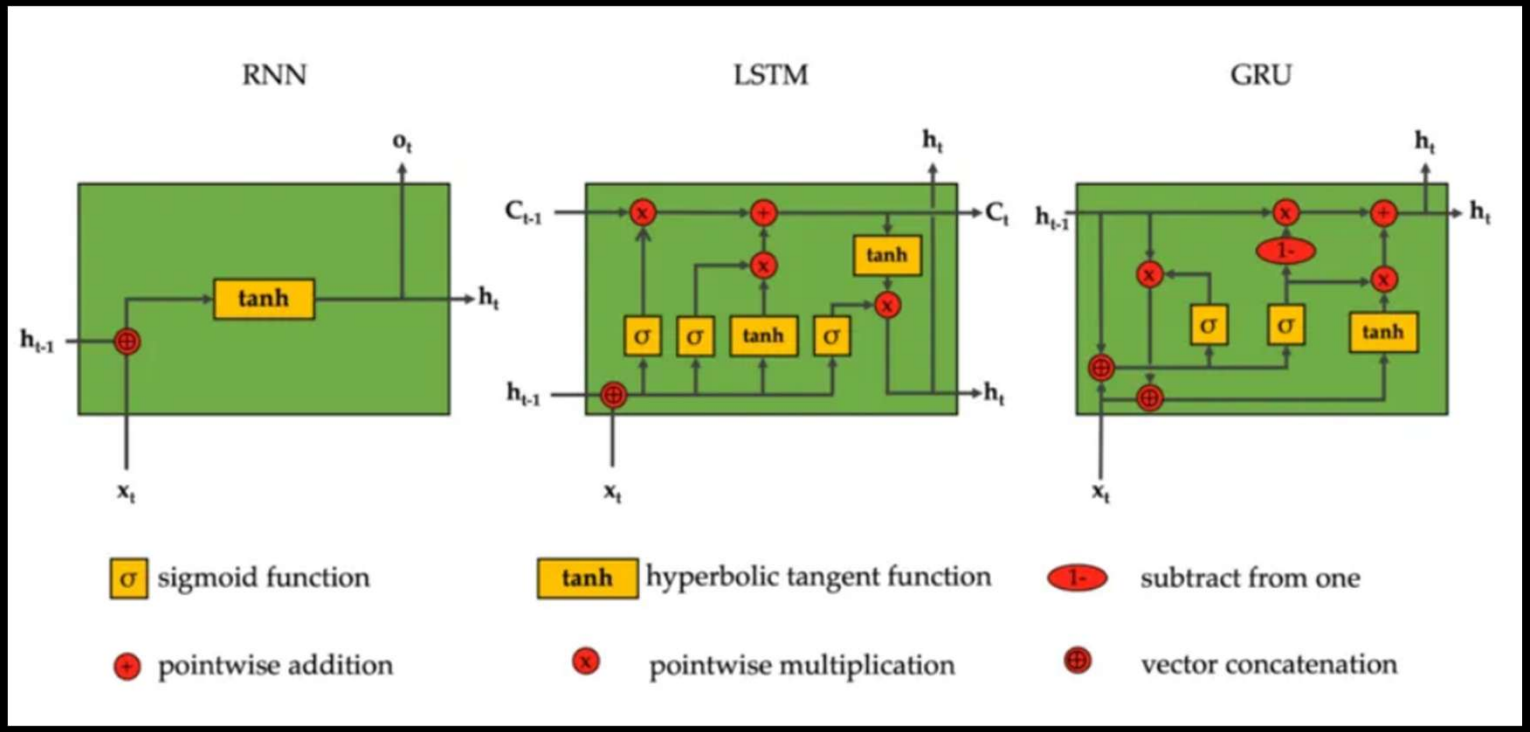
Object detection demos

https://www.kaggle.com/code/nirvanafl/comvisionproj-yolo8/notebook

https://huggingface.co/spaces/stevengrove/YOLO-World

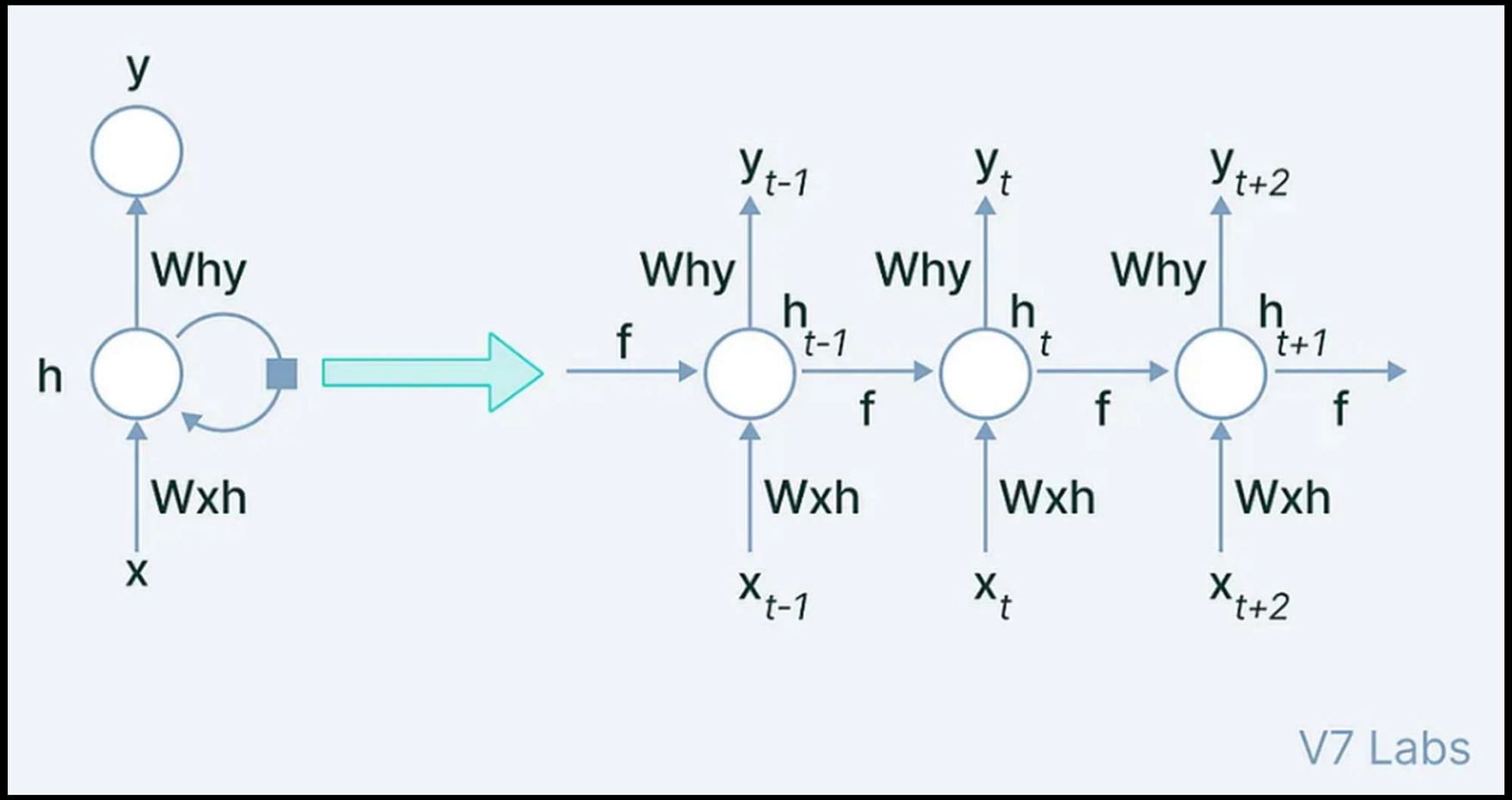
https://huggingface.co/spaces/stevengrove/YOLO-World/blob/main/README.md

RNN, LSTM and GRU



- Concept of limited memory (state) enabling sequential data processing
- Examples: Timeseries analysis, text processing, Audio processing
- Enabled NLP (Natural Language Processing) use-cases

Recurrent Neural Networks (RNN)



RNN- Recurrent connection

Recurrent Connection: This is a crucial feature of RNNs. It is a set of weights and connections that loop back to the hidden layer from itself at the previous time step. This loop allows the RNN to maintain and update its hidden state as it processes each element in the sequence. The recurrent connection enables the network to remember and utilize information from the past.

RNN Applications

- Healthcare:
  Sequential analysis of disease progression, key metrics deterioration

- Stock Market
  Analyse stock data over time

- Text

- Audio

Challenges with RNN – vanishing gradient

The vanishing gradient problem occurs when training deep Recurrent Neural Networks (RNNs)

Gradients **shrink exponentially** as they are propagated back through time.

This makes it difficult for the model to learn long-term dependencies because earlier layers receive negligible updates during backpropagation.

Vanishing gradient example

Let's say we are building an RNN model to predict the likelihood of a diabetic patient developing complications (e.g., kidney failure) based on their historical medical records.

We have sequential health data for patients collected over 10 years, including:

- Blood sugar levels
- Blood pressure readings
- Medication adherence
- Lab test results
- Lifestyle habits (exercise, diet)

The goal is to predict whether the patient will develop kidney failure in the next 2 years.

Vanishing gradient example contd.

Suppose an early symptom (e.g., consistently high blood sugar levels 5 years ago) is a key predictor of kidney failure.

The RNN needs to remember this early symptom while processing later medical records.

However, during training:
The error signal propagates back from the final prediction (kidney failure or not) to the earlier time steps.
If the model is deep, the gradients from the final output shrink exponentially before reaching the early time steps. (multiplication of fractions N times converges to 0)
The model fails to learn the relationship between early high blood sugar levels and kidney failure.

As a result, the RNN relies only on recent symptoms, ignoring long-term trends, leading to poor accuracy.

Exploding gradient

The exploding gradient problem happens when the gradients in a Recurrent Neural Network (RNN) grow exponentially during backpropagation. This results in unstable weight updates, making the model unable to learn effectively.

Why does this happen?

In an RNN, gradients are passed backward through time.
If the weight updates keep multiplying by large values, they grow exponentially.
This leads to very large updates, making the model's parameters explode to extremely high values.

When this happens, the model’s loss function fluctuates wildly and fails to converge.

Exploding gradient – real world example

Imagine we are building an RNN model to predict the likelihood of sepsis (a severe infection) in ICU patients based on their hourly vital signs:

Dataset:
Each patient has hourly health records for 48 hours, including:

- Heart Rate
- Blood Pressure
- Oxygen Levels
- Temperature
- White Blood Cell Count

Problem:
The RNN must analyze hourly data sequences and predict whether the patient will develop sepsis in the next 6 hours.
If we use ReLU as the activation function, the model starts learning but quickly encounters exploding gradients.

Exploding gradient – real world example contd.

Exploding Gradient Happens in This Case:

Early time steps (hour 1-10): Model learns normal variations in patient vitals.

Middle time steps (hour 20-30): Slightly larger weight updates occur as the model detects risk patterns.

Later time steps (hour 40-48): The RNN keeps multiplying gradients backward in time.

Backpropagation amplifies the problem:
Since ReLU outputs unbounded values, each time step multiplies the previous gradients, making them exponentially large.Loss function diverges:

The model fails to learn meaningful trends and starts producing NaN (Not-a-Number) losse

Choosing the right activation function

Activation	Where to Use	Pros	Cons
ReLU	RNNs (Shallow or Deep)	Faster training, helps mitigate vanishing gradients	Exploding gradients possible, no negative values
tanh	RNNs (Classic) & LSTMs	Keeps values balanced, better for sequential data	Slower training, vanishing gradients in deep networks
sigmoid	Rarely Used	Useful for probability-based tasks	Strong vanishing gradient issue

Hidden size optimization

The number of hidden units (neurons in the hidden layer) is a hyperparameter that directly affects the model's memory capacity, performance, and computational cost.

While there is no theoretical limit, there are practical constraints based on:

- Computational resources (GPU/CPU/RAM)
- Model performance (overfitting vs underfitting)
- Training time

Model	Max Practical Hidden Units
Vanilla RNN	50-200 (Exploding gradients issue)
LSTM / GRU	128-512 (More stable)
Deep LSTM (Stacked)	256-1024 (With strong GPUs)

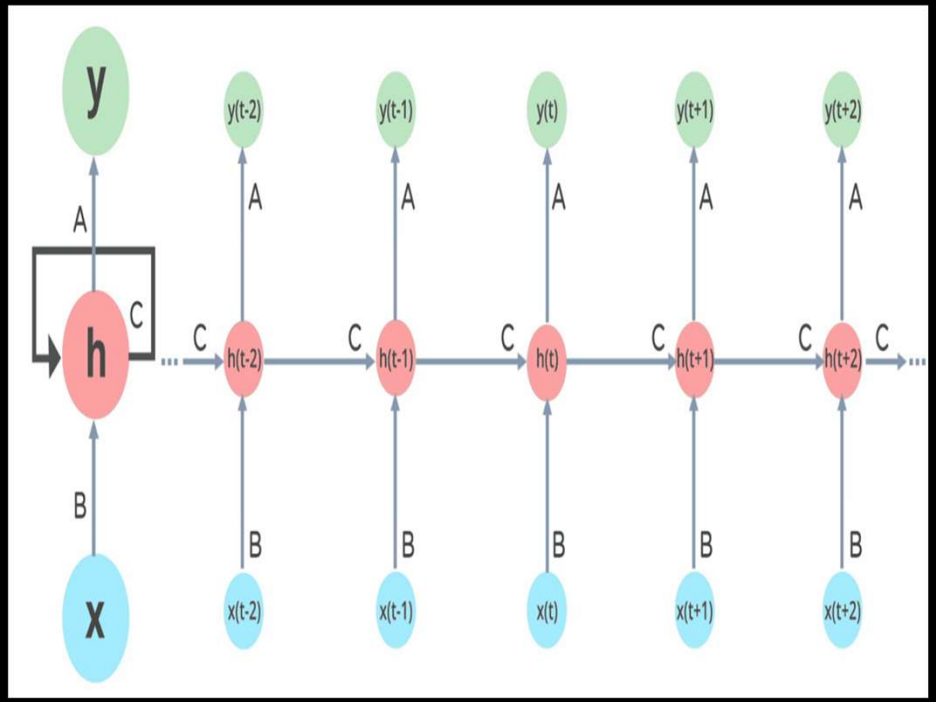
💡 LSTMs and GRUs allow for higher hidden sizes than Vanilla RNNs because they prevent vanishing gradients.

Recurrent Neural Networks (RNN)

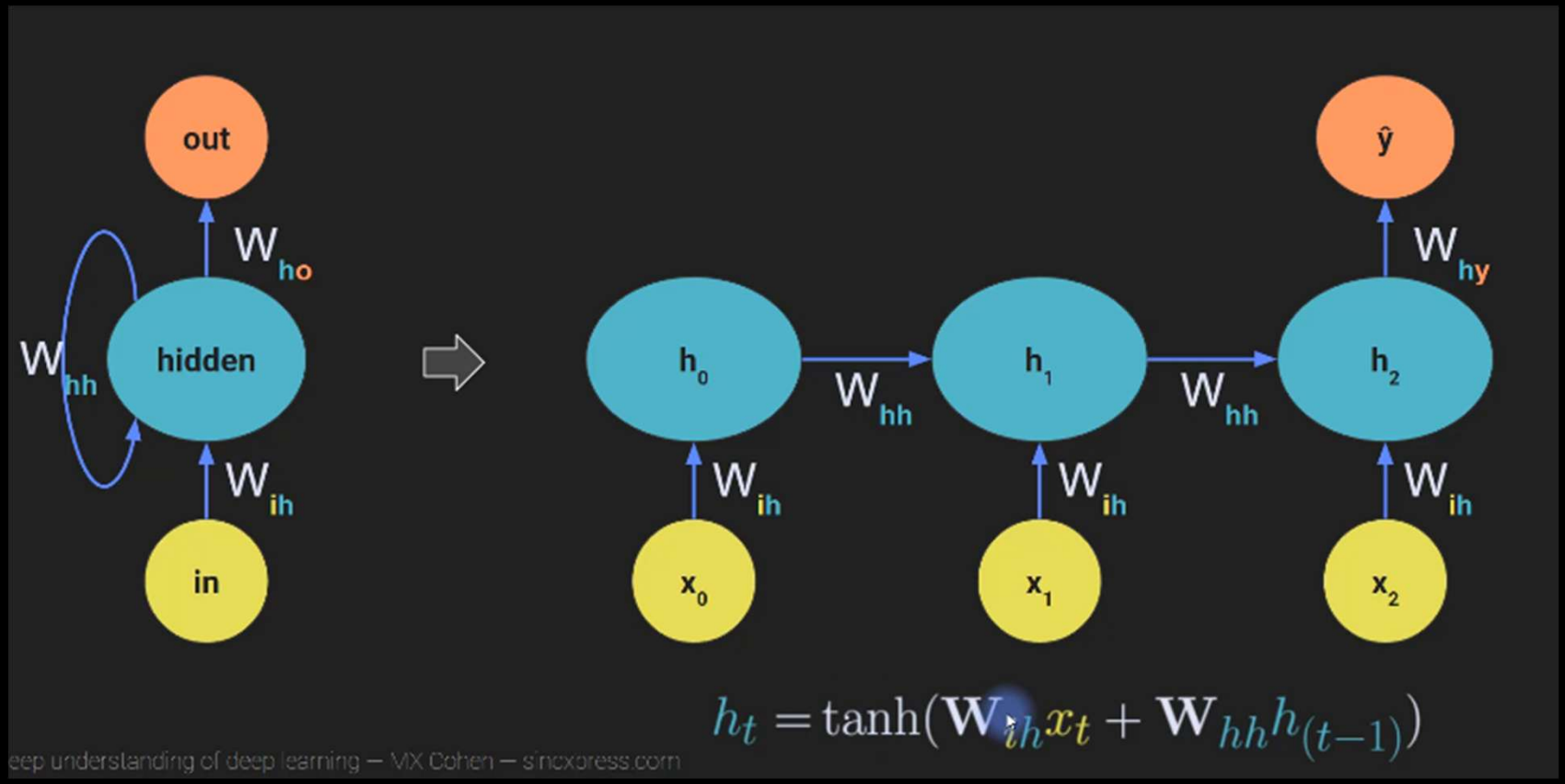
Recurrent neural networks (RNNs) are a type of neural network that is well-suited for processing sequential data.

This is because RNNs have an internal state that allows them to remember information from previous inputs.

This makes them ideal for tasks such as **natural language processing (NLP)**, where the meaning of a word or phrase can depend on its context



Recurrent Neural Networks (RNN)



h0 = hidden state at time t0
h1 = hidden state at time t1
It is cumulative – combination of all previous hidden states is reflected in the next layer

Applications of RNN

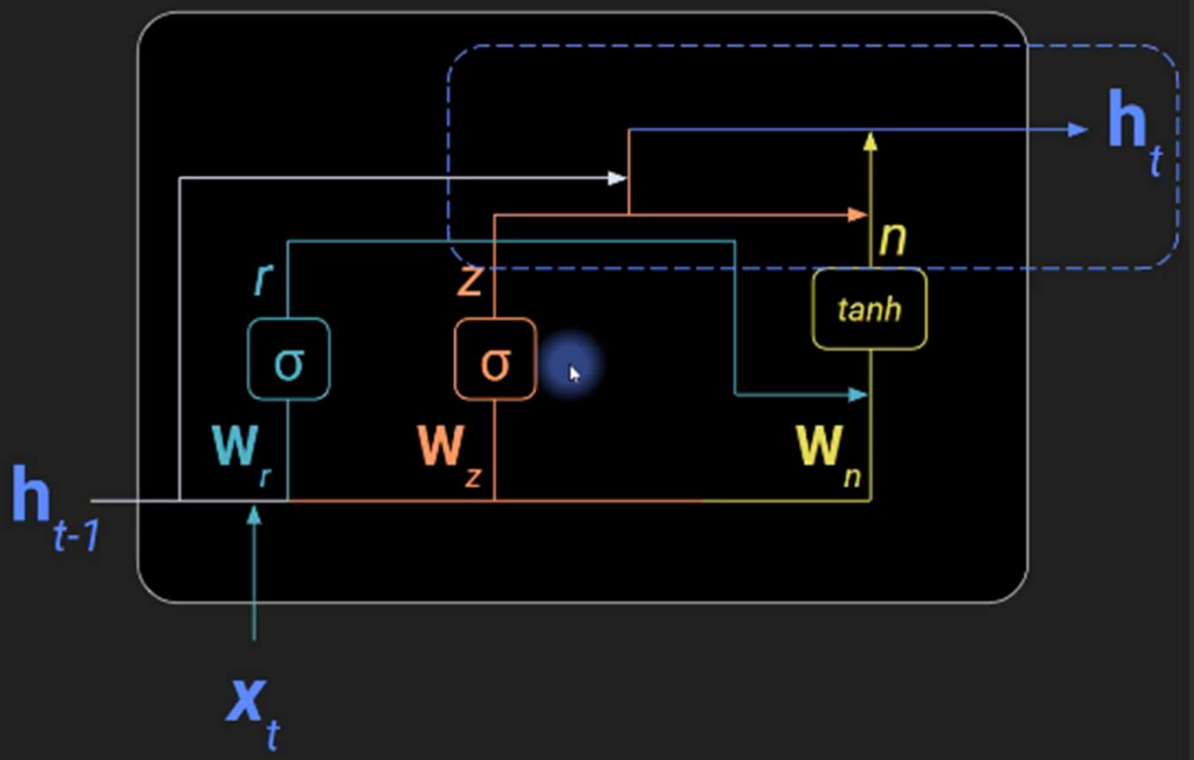
	x - input		y - output
Speech recognition		⟹	"Fuzzy Wuzzy was a bear. Fuzzy Wuzzy had no hair."
Music generation	Ø	⟹	
Sentiment classification	"Decent effort. The plot could have been better."	⟹	
DNA sequence analysis	ACTGTACCCATGTGACTGCCC	⟹	ACTGTACCCATGTGACTGCCC
Machine translation	"El que no arriesga, no gana."	⟹	"If you don't take risks, you cannot win."
Video activity recognition		⟹	Running
Name entity recognition	"Ygritte says Jon Snow knows nothing."	⟹	"Ygritte says Jon Snow knows nothing."

Gated Recurrent Unit (GRU)

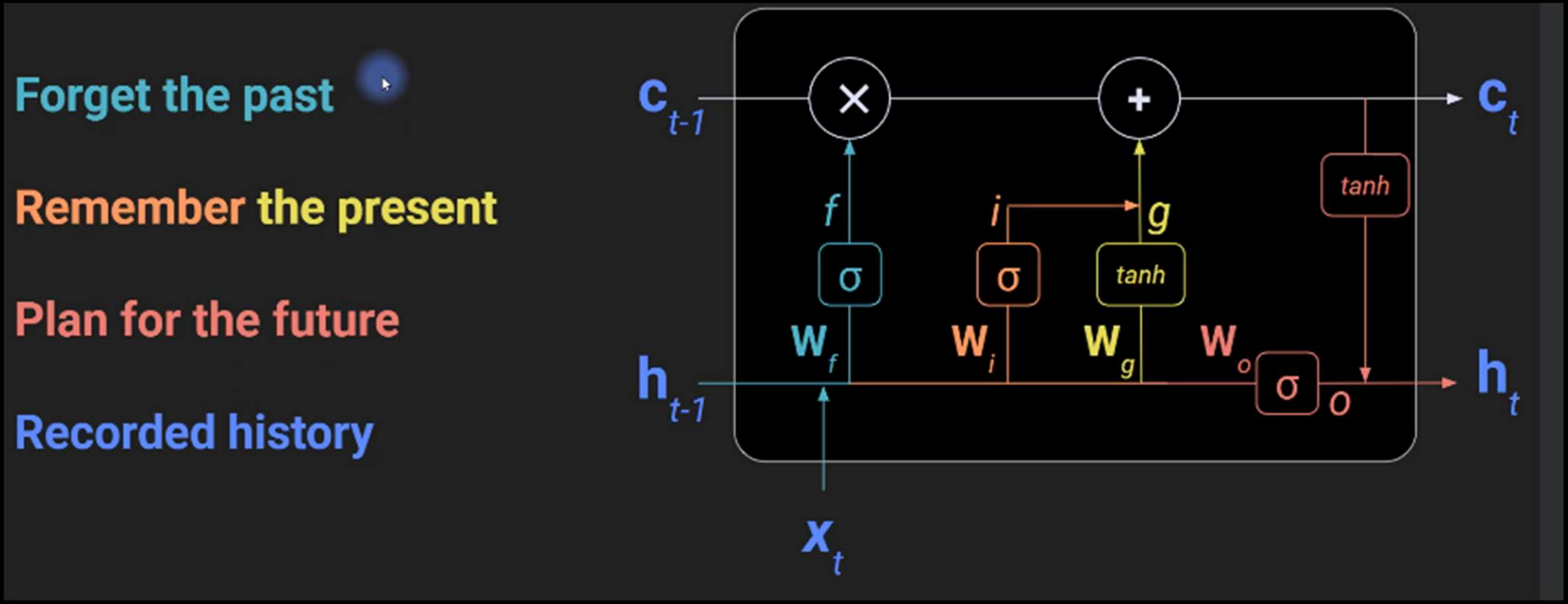
$$h_t = (1 - z_t)n_t + z_t h_{t-1}$$

Output (hidden state)

Weighted combination of to-remember new state (*n*) and to-forget old state (h_{t-1}).



Long Short Term Memory (LSTM)



H = hidden state from previous time point
C = cell: preserves long term memory traits via c

In practice LSTMs typically handle:
~100 to 500 tokens effectively
beyond ~1000 → performance degrades significantly

When to use what?

Standard RNN: Simple, fewest parameters. Good for sequences with short patterns.

GRU: More complex than standard RNNs, but simpler than LSTMs. Trains fast and highly scalable, but needs more data than RNNs.

LSTM: Most complex, requires more training data and computation time. Explicitly includes a memory trace. Good for longer sequences or complex patterns.

Sentiment Analysis with LSTM

For example:
"Patient is anxious and reports severe pain“ → negative
"Patient feels better today" → positive

Input to LSTM when training:

- Text cannot directly be fed to the model – it needs numbers
- Words will nee to be converted into token_ids
- Unique words or sub-words (tokens) from the training data will form the vocabulary of the model

```
"Patient is anxious and reports severe pain"
→ [45, 12, 876, 9, 221, 334, 98]
```

LSTM expects fixed length sequences, so the notes will be padded as needed

Feature Extraction - Embeddings

Each token will be converted to an embedding of N dimension, so that features of the token/word can be undersood by the model.

An Embedding layer will do it.

If Embedding has 300 dimensions, the input data shape will be:

```
32 patient notes
100 tokens per note
300 features per token
```

(batch_size, timesteps, features)

In this case:
(32, 100, 300)

A 3-dimensional matrix, aka **tensor**

Output layer

Each token will be converted to an embedding of N dimension, so that features of the token/word can be undersood by the model.

An Embedding layer will do it.

If Embedding has 300 dimensions, the input data shape will be:

```
32 patient notes
100 tokens per note
300 features per token
```

(batch_size, timesteps, features)

In this case:
(32, 100, 300)

A 3-dimensional matrix, aka **tensor**

Transfer Learning

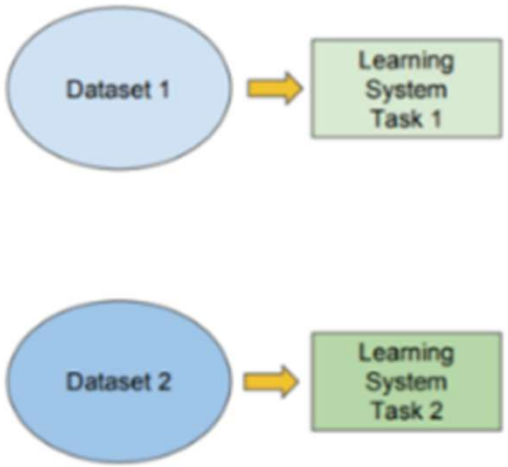
What is Transfer Learning?

Transfer learning is a machine learning technique
where a **model developed for one task** is
reused (either as-is or with some fine-tuning)
for a **different** but related task

Transfer Learning

Traditional ML

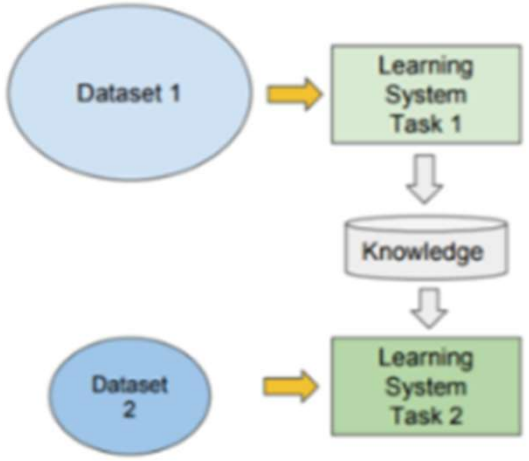
- Isolated, single task learning:
  - Knowledge is not retained or accumulated. Learning is performed w.o. considering past learned knowledge in other tasks



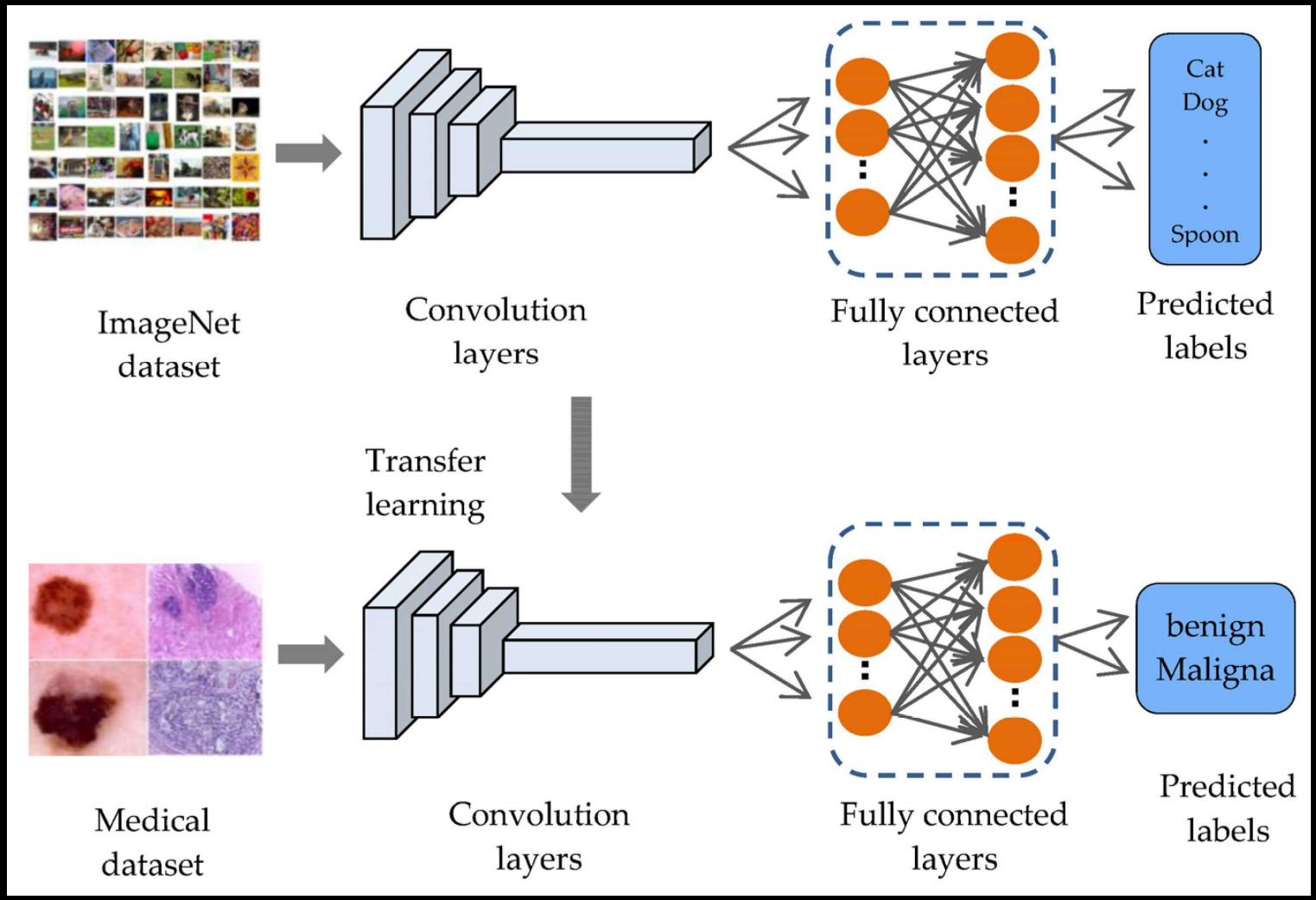
vs

Transfer Learning

- Learning of a new tasks relies on the previous learned tasks:
  - Learning process can be faster, more accurate and/or need less training data

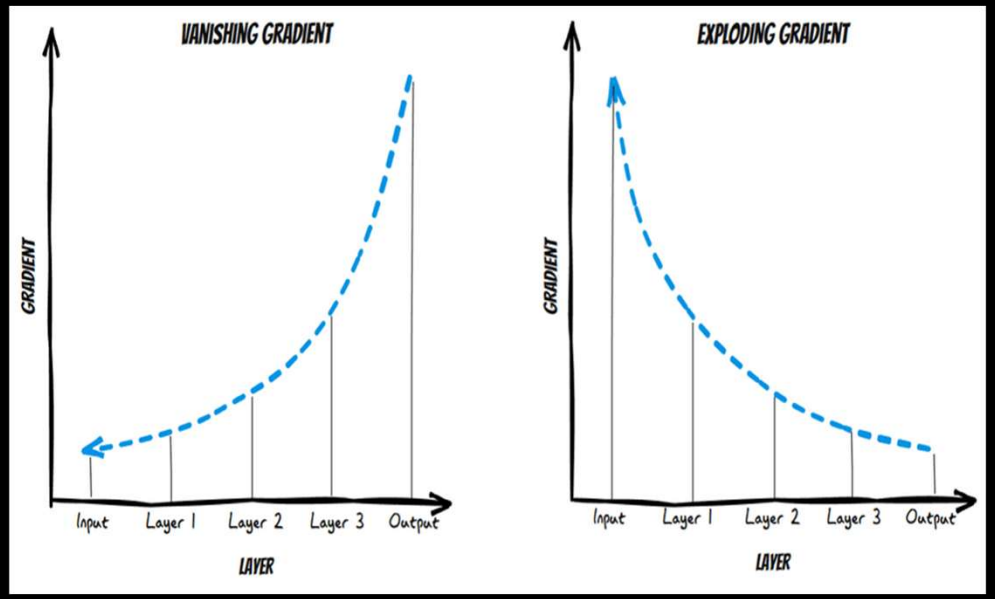


Transfer Learning



Limitations of RNN, LSTM

- Limited ability to process context, limited memory
  Text is sequential data, hence ability to process context is important
- Suffered from problems such as vanishing gradient, exploding gradient
- Computationally intensive to train

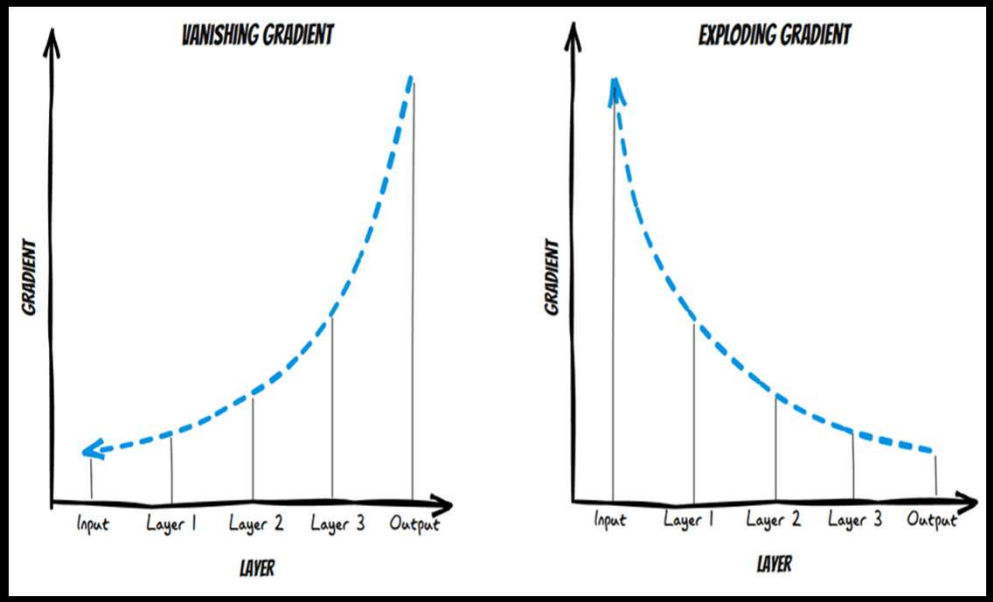


Limitations of RNN, LSTM

- Limited ability to process context, limited memory
  Text is sequential data, hence ability to process context is important

- Suffered from problems such as vanishing gradient, exploding gradient

- Takes lot of time to train and resources to train



Vanishing / Exploding gradients

🧠 The core problem: repeated multiplication over time

In an RNN, the hidden state evolves like:

```
h_t = f(W_h · h_{t-1} + W_x · x_t)
```

During **backpropagation through time (BPTT)**, gradients flow *backward* through all time steps.

👉 That means:

- Gradients are multiplied **again and again** across steps
- Roughly like:

```
Gradient ∝ (W_h)^T × (W_h)^T × ... × (W_h)^T   (T times)
```

```
0.8 × 0.8 × 0.8 × ... → 0
```

```
1.2 × 1.2 × 1.2 × ... → ∞
```

LSTMs are limited not by hard sequence length but by a fixed-size memory and sequential information compression, which causes long-range information to fade or become inaccessible over time.

Limited Context

LSTMs struggle to distinguish between bank" in **river bank** vs. **I went to the bank to deposit money**

- **Lack of Word Context Awareness**
  LSTMs use embeddings (e.g., Word2Vec, GloVe).
  That assign a single vector representation per word, meaning "bank" has the same representation in both sentences.

- **Limited Context Window**
  While LSTMs have memory, they often struggle with long-distance dependencies. If the sentence is long, the context necessary to disambiguate "bank" might be lost.

- **No Explicit Word Sense Disambiguation**
  Do not dynamically adjust word meanings based on context.

Enter Transformers!!!

Transformers are a type of neural network that have a **unique ability to recognize long-range connections within sequences**. They are particularly useful for **tasks like generating text**, as the model needs to comprehend the preceding words in order to produce the next one.

The introduction of transformers in 2018 was a groundbreaking moment for the field of natural language processing.

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* † illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after

- No compression into single vector
- Direct attention to all tokens
- No sequential bottleneck

Seminal paper published in 2017

Transformers – key capabilities

Transformers are able to learn long-range dependencies in sequences.

Transformers are able to be trained on very large datasets.

Transformers are able to be parallelized

GPT

Generative

Pretrained

Transformer

Is LSTM used still now that we have transformers?

In most NLP use-cases, transformers are now the goto option. However, LSTMs are still a **reasonable or even right choice** in some practical scenarios, especially when data, compute, latency, or explainability constraints matter.

Use case	LSTM suitability
Sentiment from short patient feedback notes	Good
Urgency classification from nurse notes	Good
Medication/symptom extraction with BiLSTM-CRF	Reasonable
ICU event-sequence prediction	Good
Long discharge-summary QA	Poor
Clinical semantic search	Poor
Summarizing patient history	Poor
Medical chatbot	Poor